

第二章

并发进程

主要内容

2.1 进程及其调度控制与死锁

2.2 UNIX的进程

2.3 中断的基本概念及UNIX中断处理（续）

2.4 UNIX时钟中断与系统调用

2.5 UNIX的进程调度状态

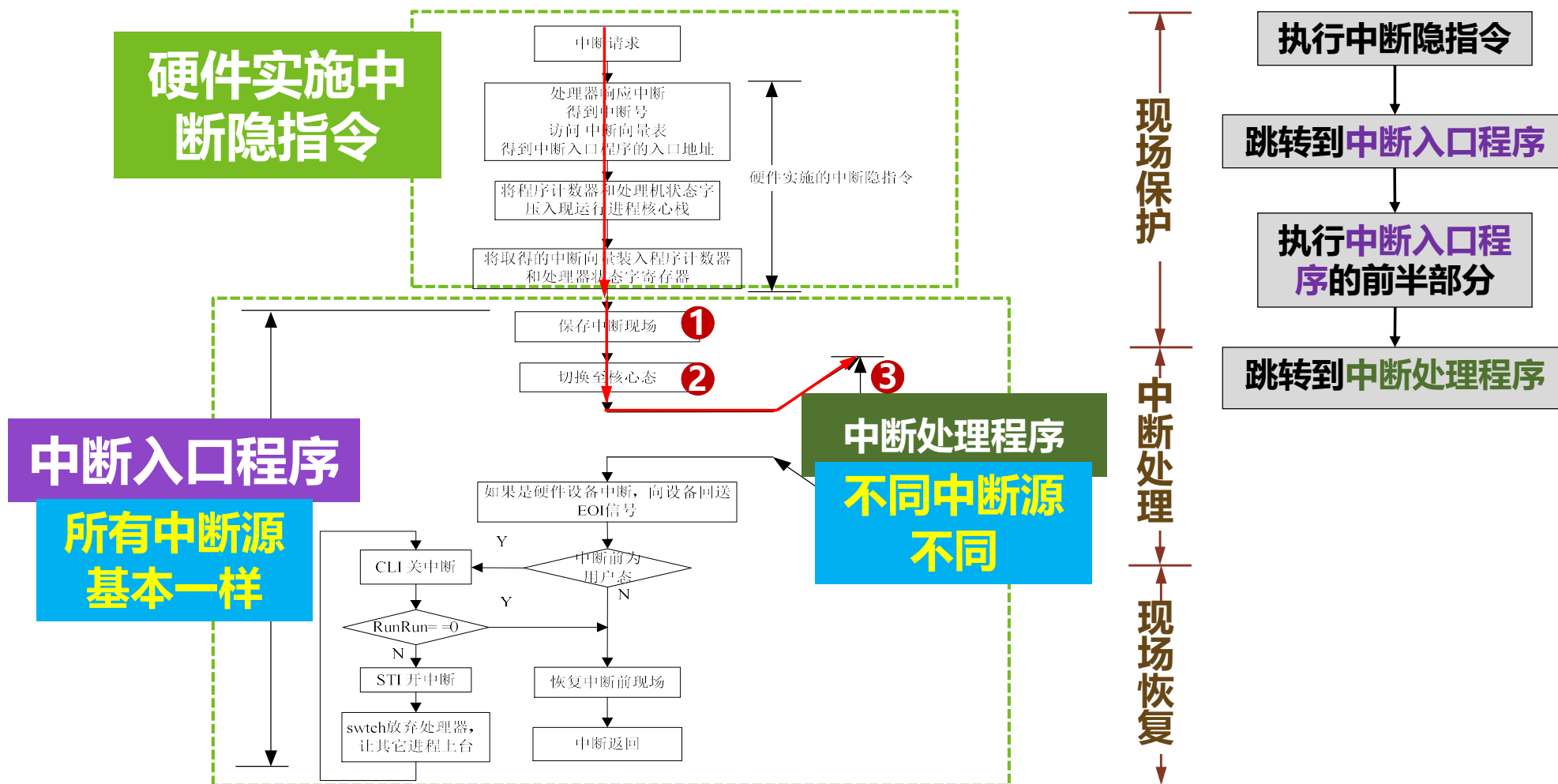
2.6 进程通讯



UNIX中断处理流程



中断入口程序



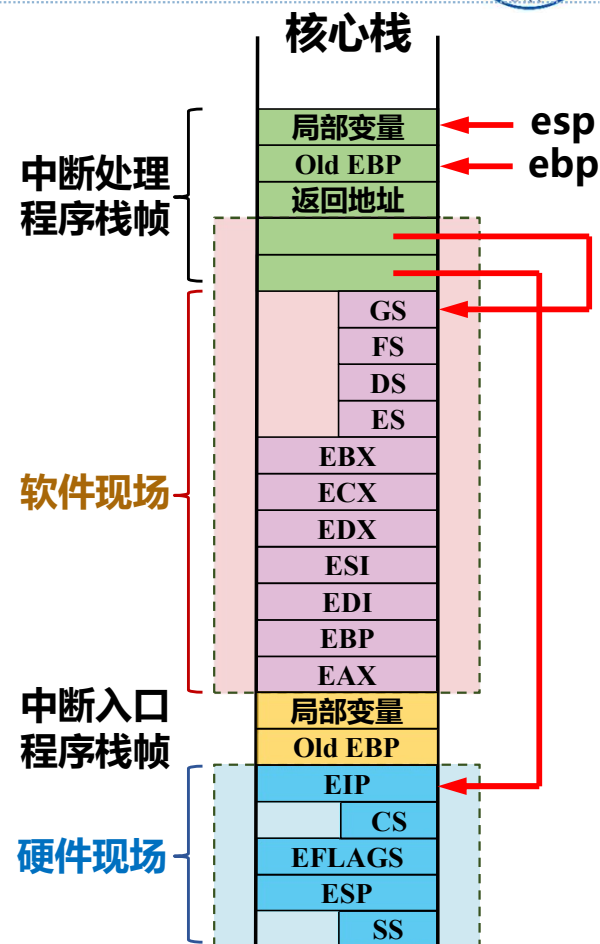
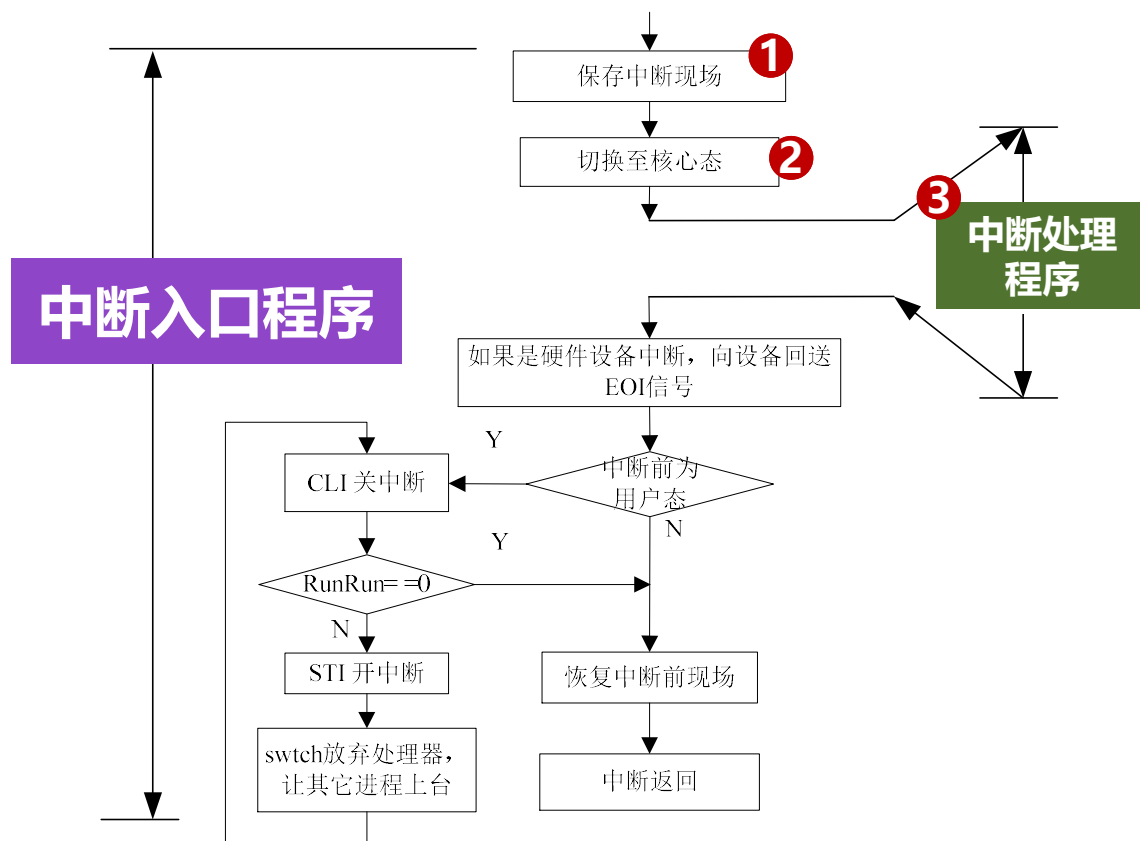


UNIX中断处理流程



中断入口程序

(3) : 调用中断处理程序



中断发生时现运行进程在用户态运行

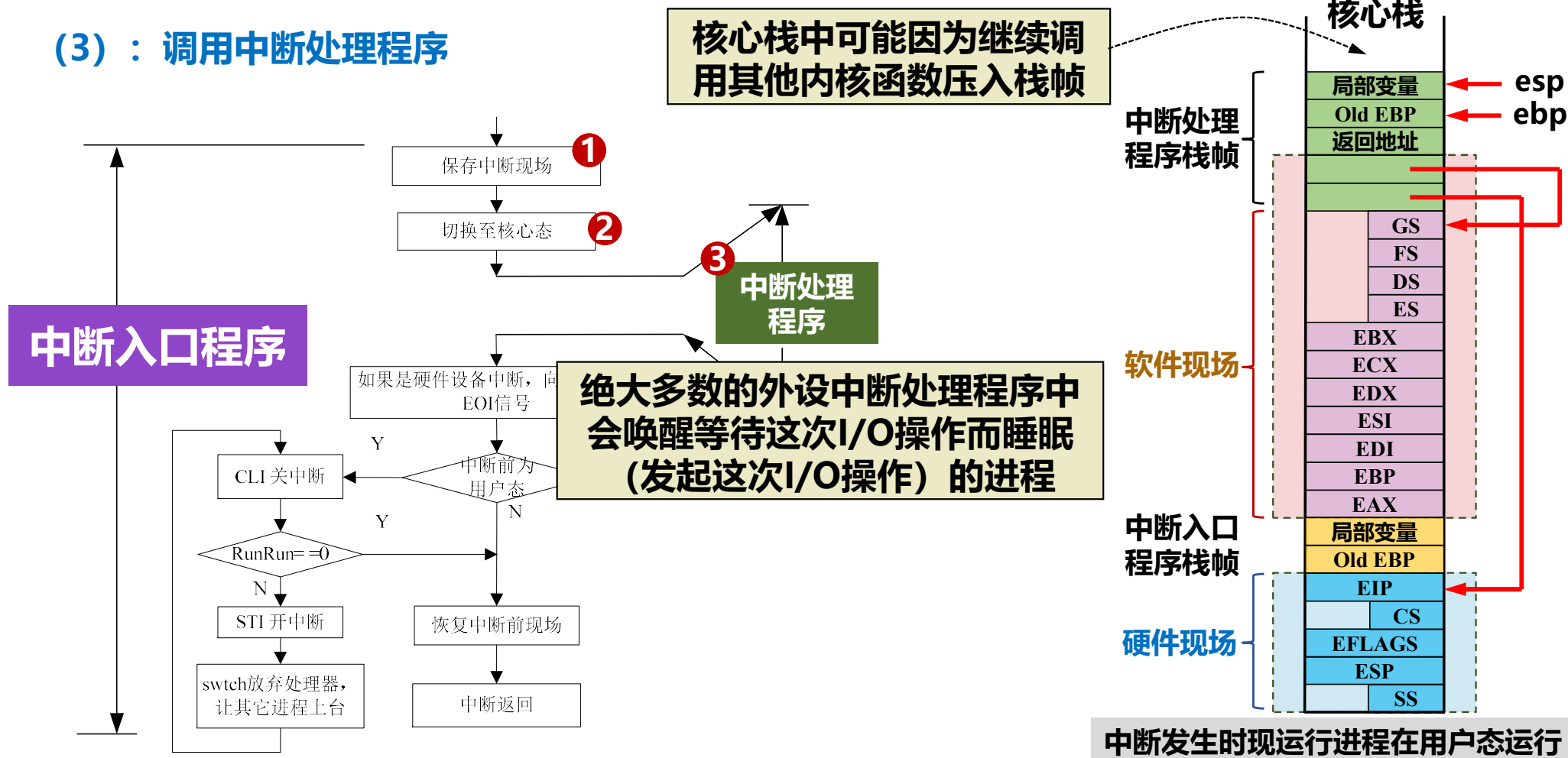


UNIX中断处理流程



中断处理程序

(3) : 调用中断处理程序





中断的基本概念

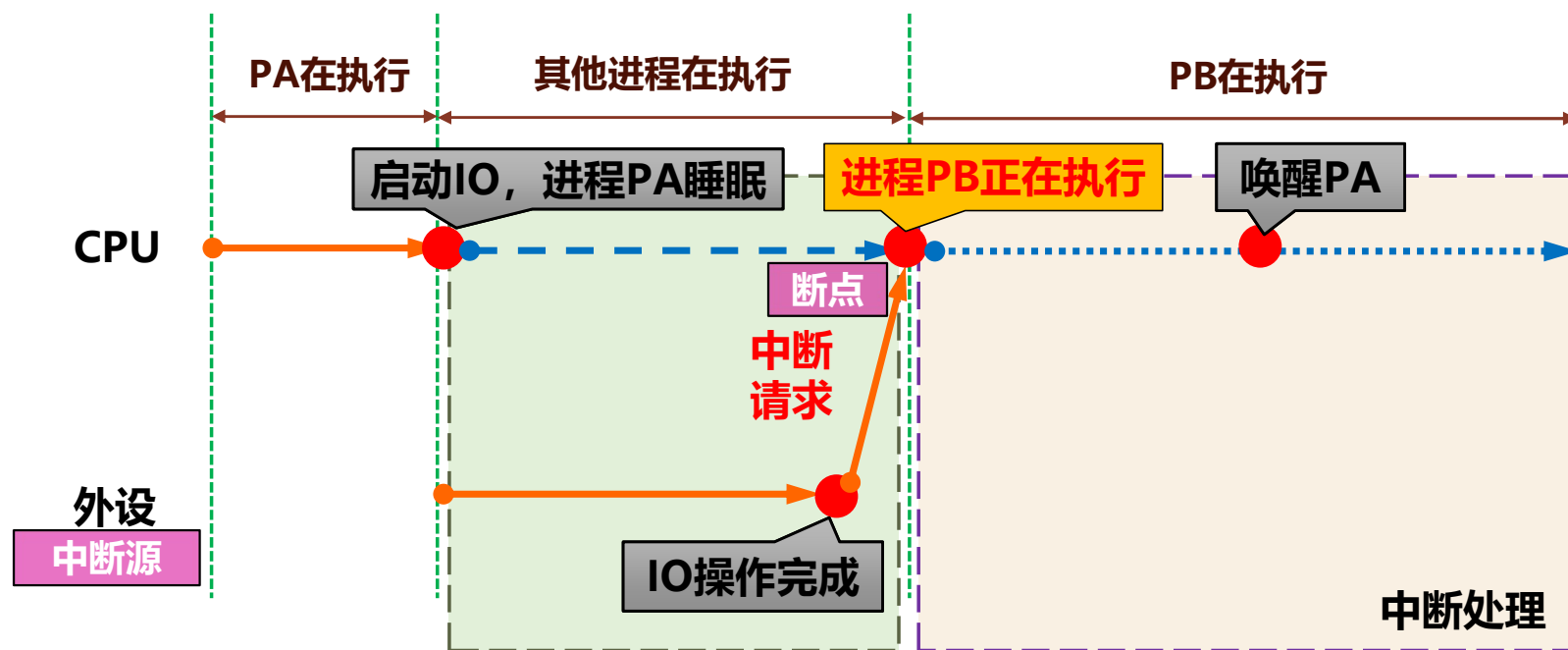
CPU不可能时刻查询外设的工作状态。。。



什么是中断

一种设备控制方式

一种外设的数据传输方式



1. 保证了CPU和设备之间的并行操作

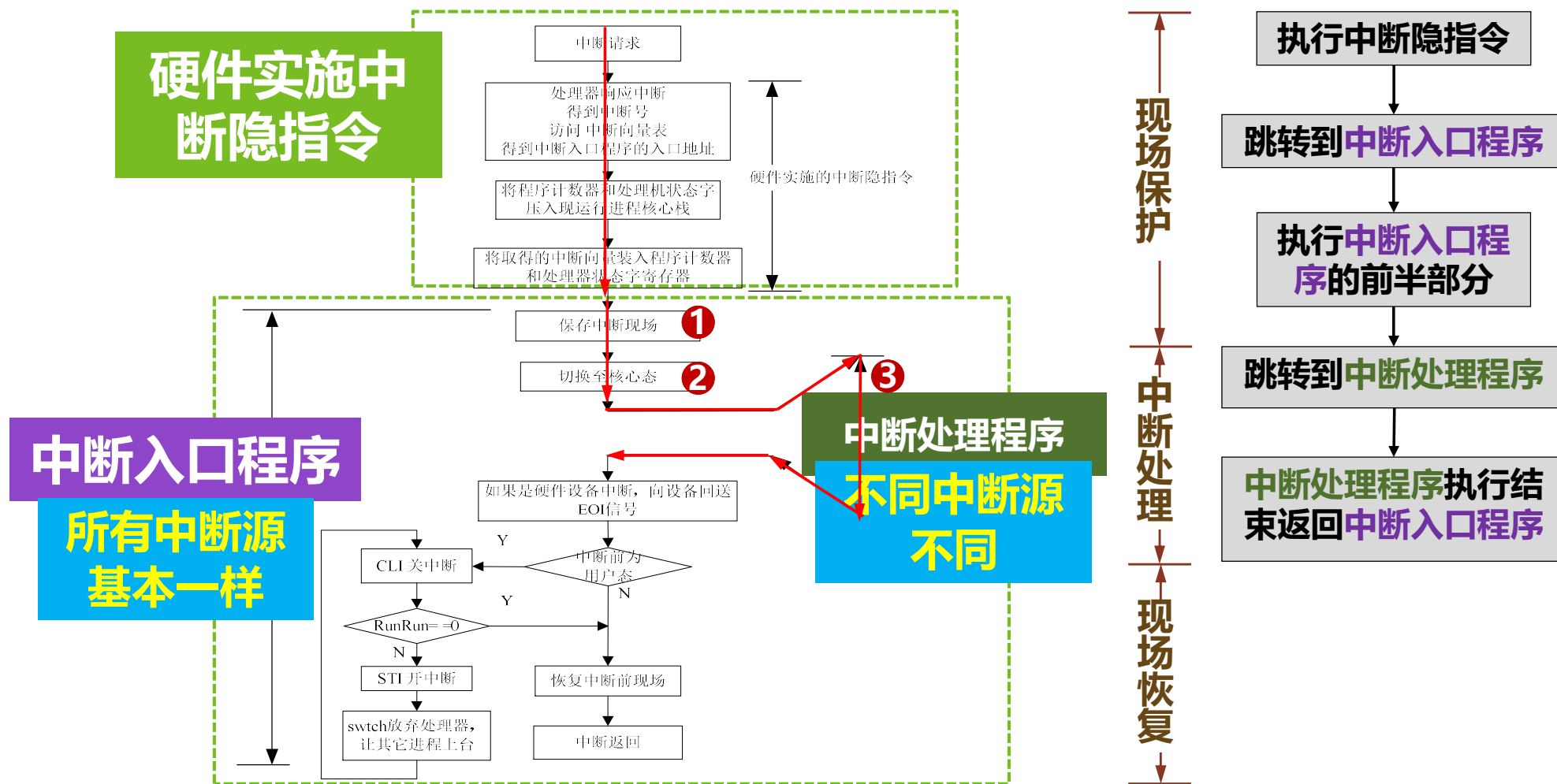
2. 提供了进程执行内核代码的机会



UNIX中断处理流程



中断处理程序



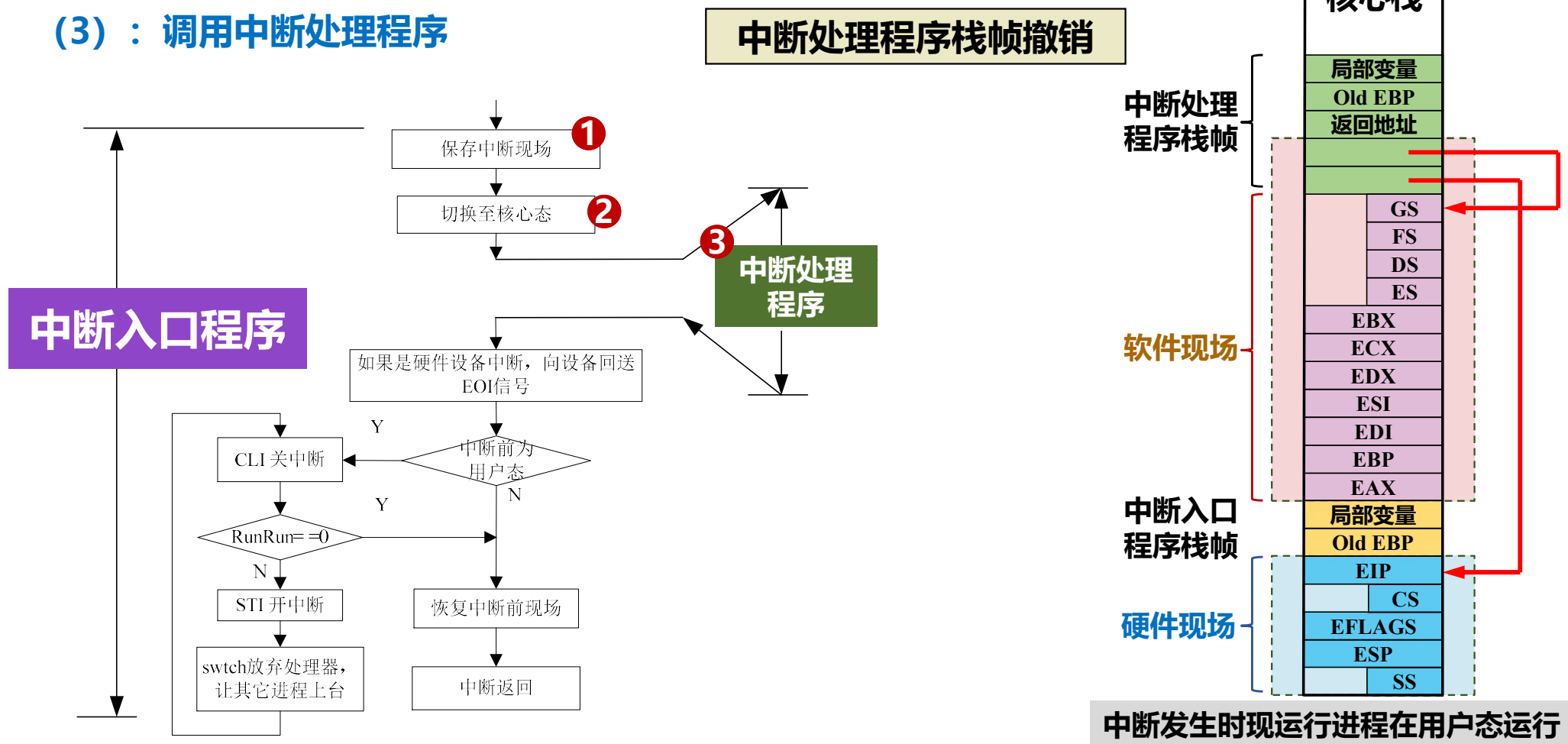


UNIX中断处理流程



中断处理程序

(3) : 调用中断处理程序

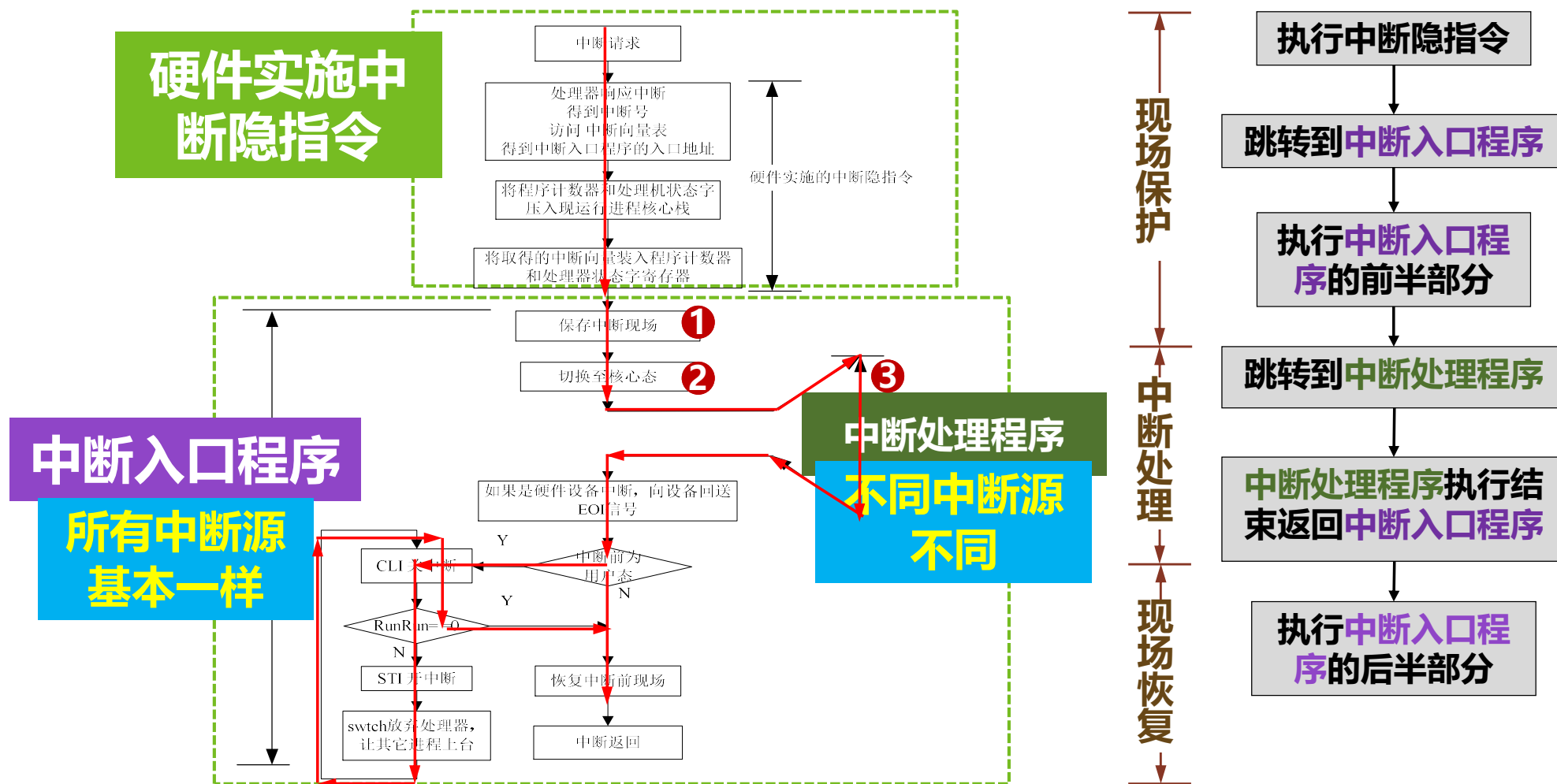




UNIX中断处理流程



中断入口程序



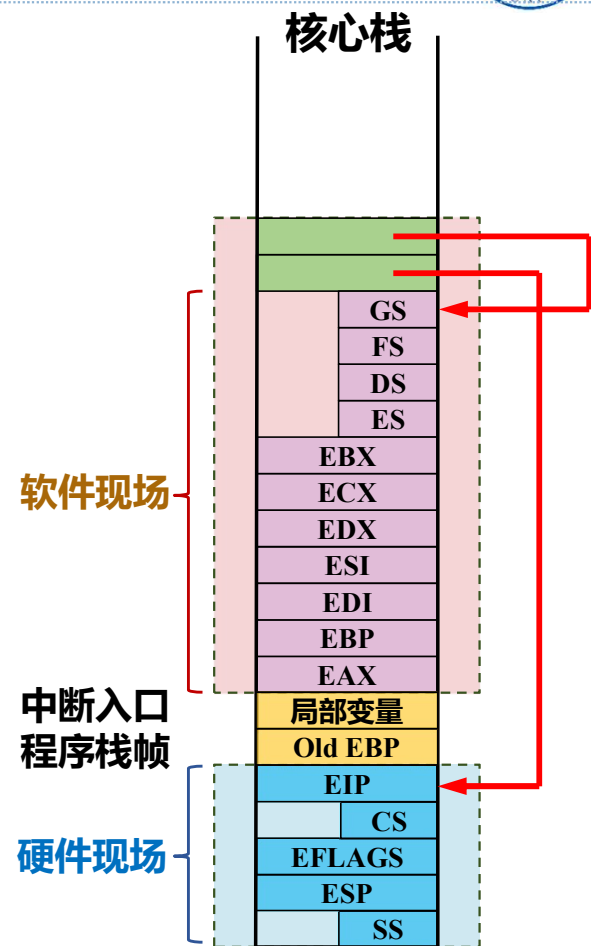
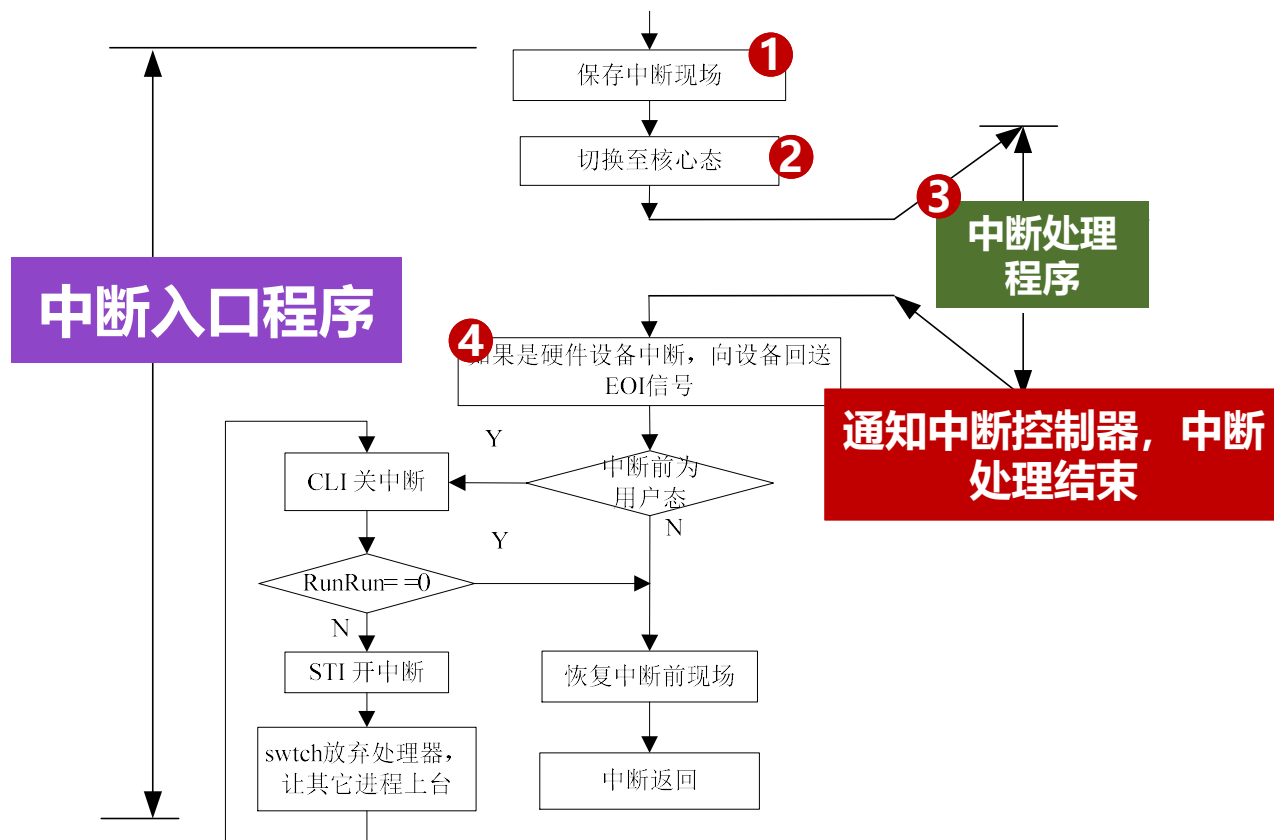


UNIX中断处理流程



(4) : 发送EOI指令

中断入口程序



中断发生时现运行进程在用户态运行

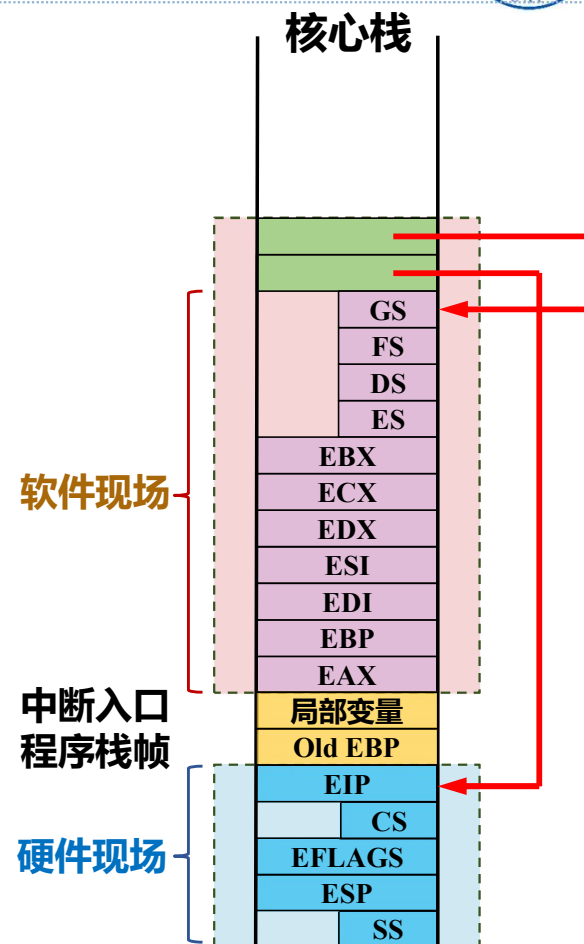
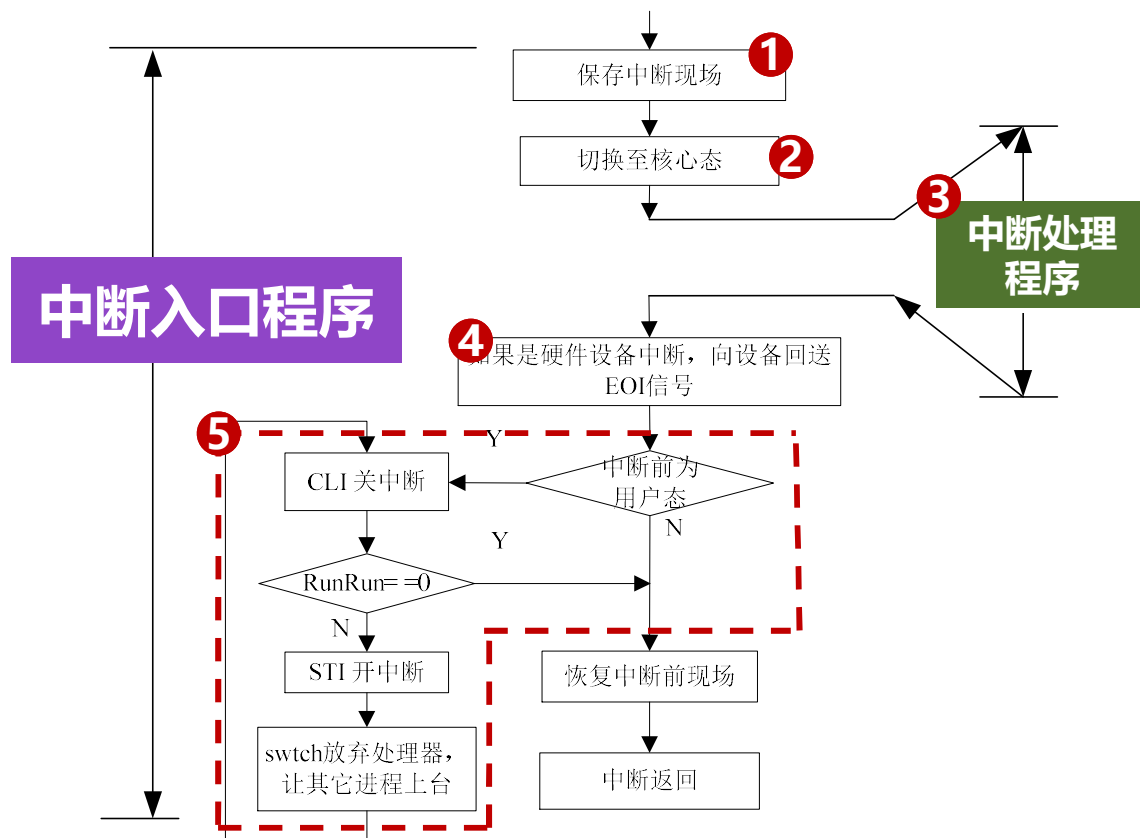


UNIX中断处理流程



(5) : 例行调度

中断入口程序



中断发生时现运行进程在用户态运行

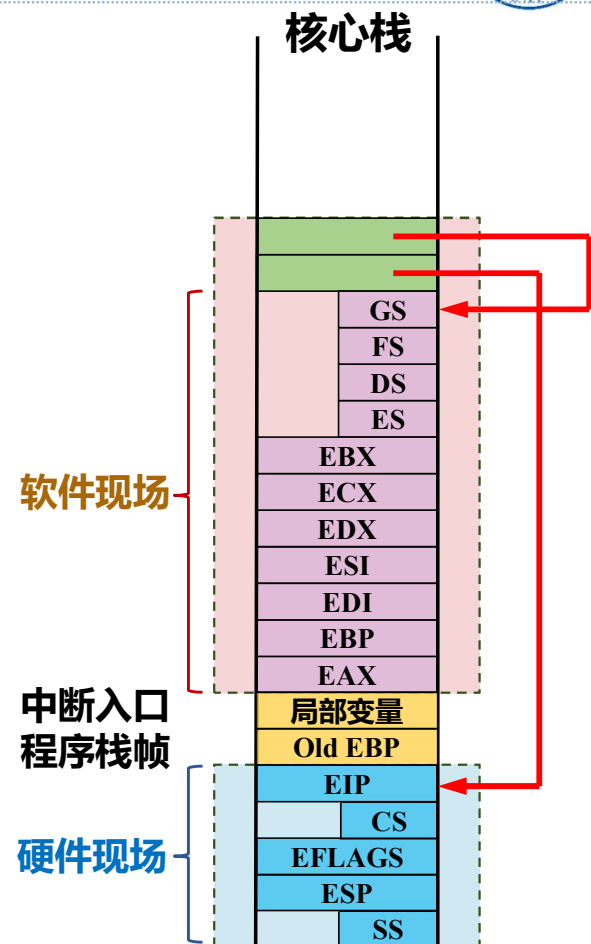
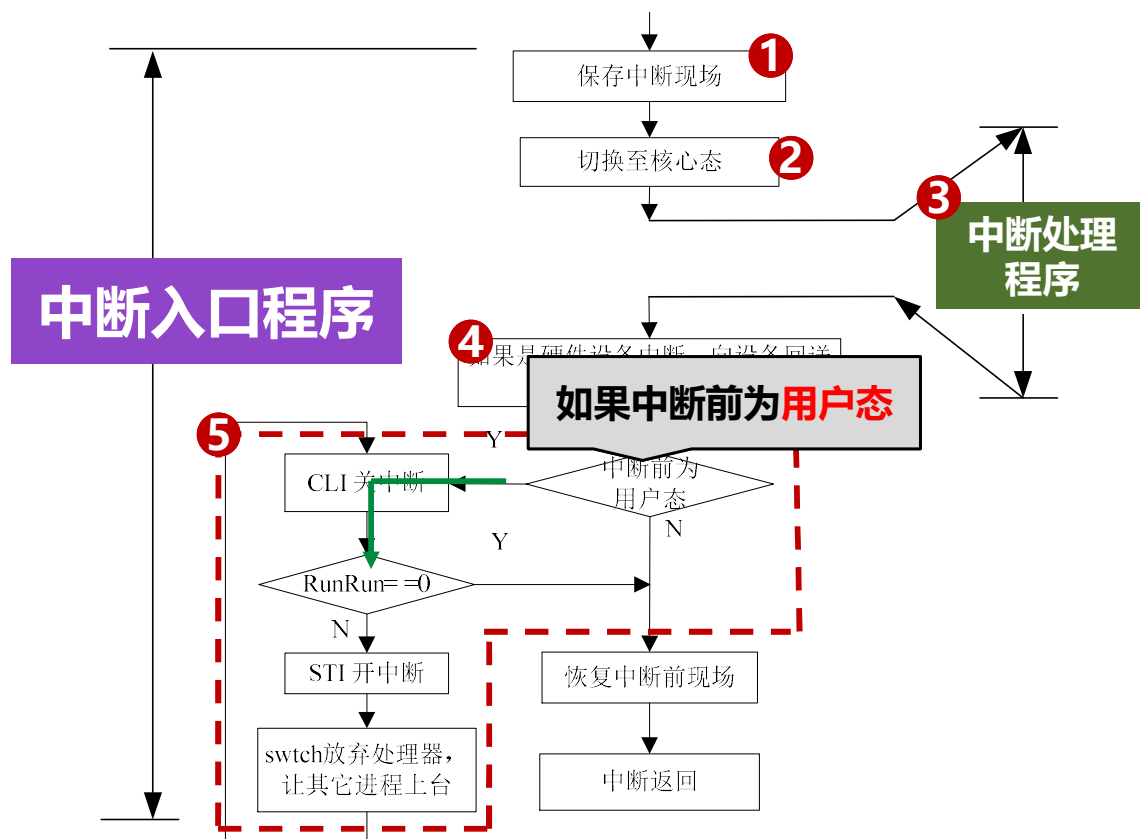


UNIX中断处理流程



(5) : 例行调度

中断入口程序



中断发生时现运行进程在用户态运行

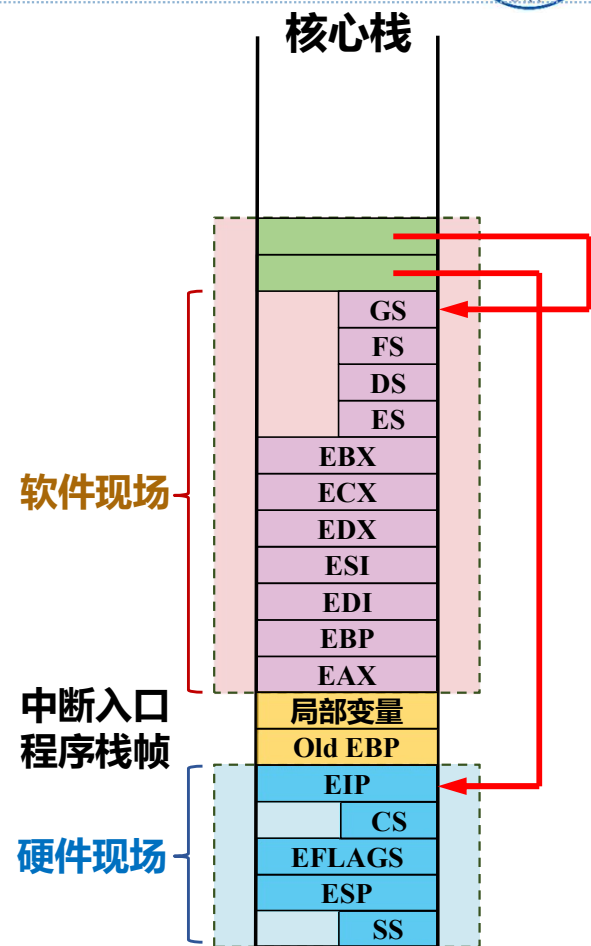
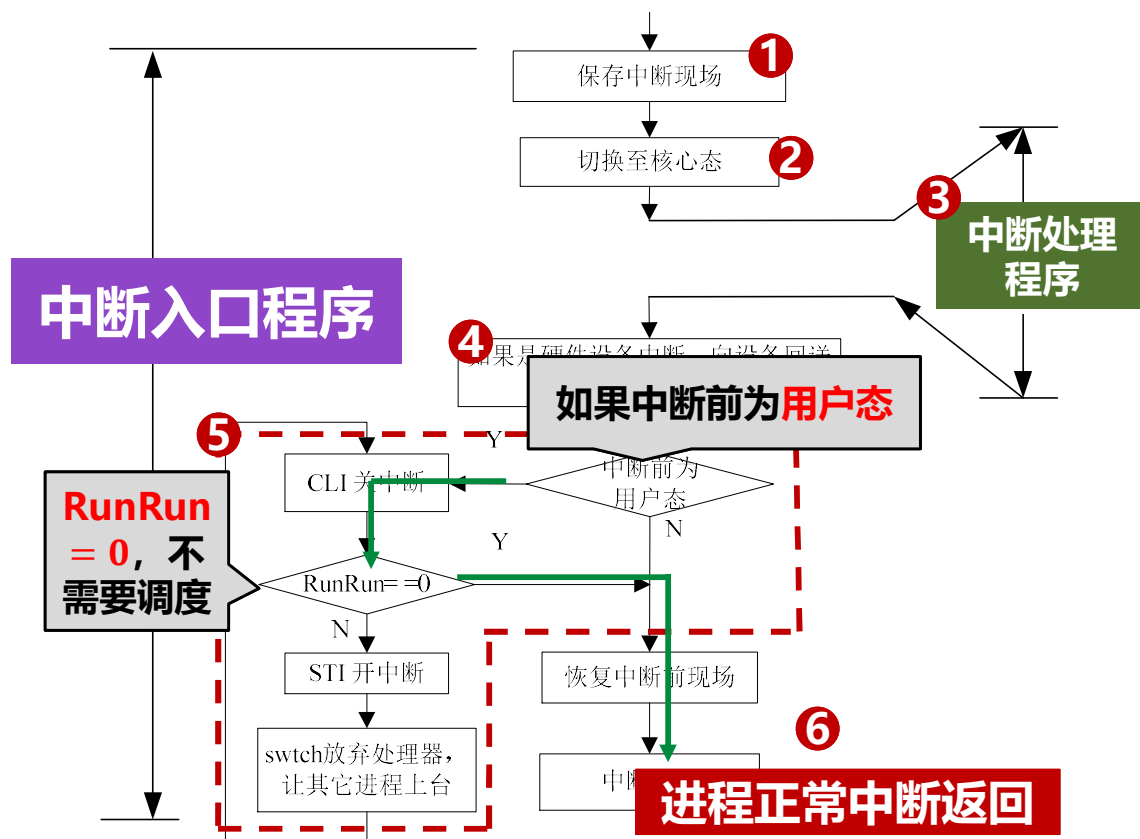


UNIX中断处理流程



(5) : 例行调度

中断入口程序



中断发生时现运行进程在用户态运行



UNIX中断处理流程



(5) : 例行调度

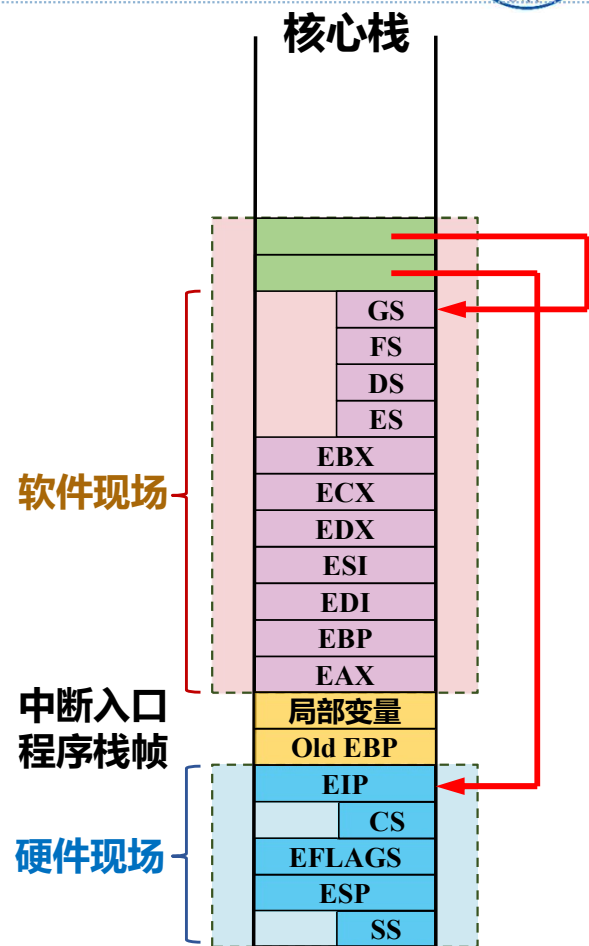
```
void Time::TimeInterruptEntrance()
{
    /* 保存中断现场 */
    SaveContext();

    /* 完全进入核心态 */
    SwitchToKernel();

    /* 调用时钟中断处理程序 */
    CallHandler(Time, Clock);

    /* 例行调度 ⑤ */
    .....

    /* 恢复现场 */
    RestoreContext();
    /* 手工销毁栈帧 */
    Leave();
    /* 退出中断 */
    InterruptReturn();
}
```



中断发生时现运行进程在用户态运行



UNIX中断处理流程



(6) : 撤销栈帧

```
void Time::TimeInterruptEntrance()
{
    /* 保存中断现场 */
    SaveContext();

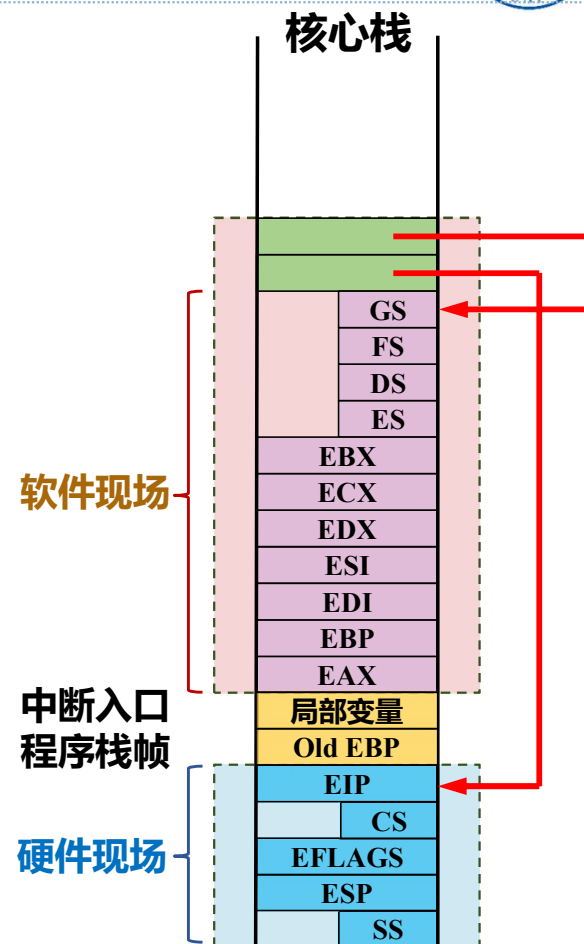
    /* 完全进入核心态 */
    SwitchToKernel();

    /* 调用时钟中断处理程序 */
    CallHandler(Time, Clock);

    /* 例行调度 */
    .....

    /* 恢复现场 */
    RestoreContext();
    /* 手工销毁栈帧 */
    Leave();
    /* 退出中断 */
    InterruptReturn();
}
```

⑥



中断发生时现运行进程在用户态运行



UNIX中断处理流程



(6) : 撤销栈帧

```
void Time::TimeInterruptEntrance()
{
    /* 保存中断现场 */
    SaveContext();
    /* 完全进入核心态 */
}
```

一组汇编指令恢复软件现场
IRET指令恢复硬件现场
(从核心栈中将相应的值重新装回寄存器中)

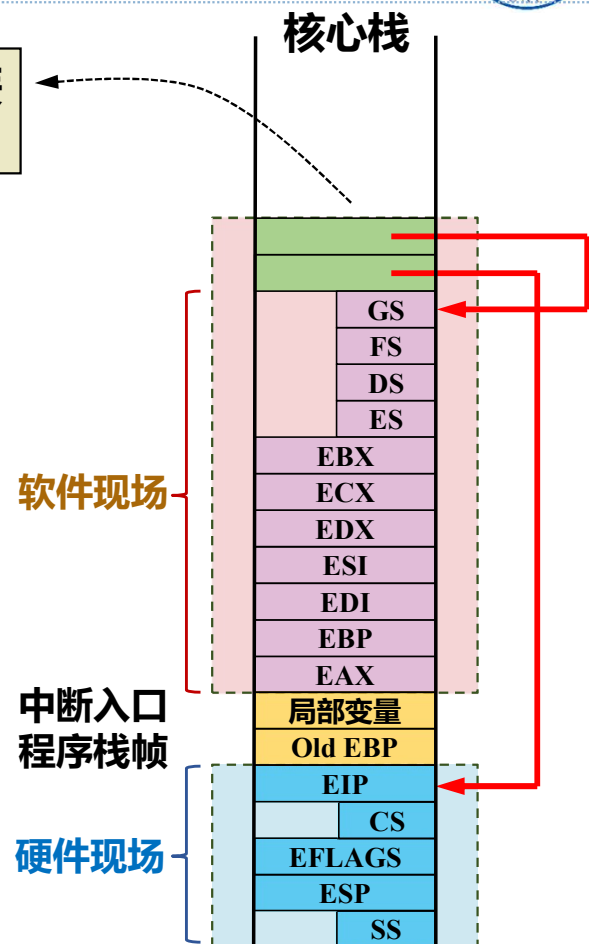
```
/* 恢复现场 */
RestoreContext();
/* 手工销毁栈帧 */
Leave();
/* 退出中断 */
InterruptReturn();
}
```

⑥

从核心栈中将保存的值装回CPU内的寄存器

恢复中断前现场

```
#define RestoreContext()
#define Leave()
```



中断发生时现运行进程在用户态运行



UNIX中断处理流程



(6) : 撤销栈帧

```
void Time::TimeInterruptEntrance()
{
    /* 保存中断现场 */
    SaveContext();
```

```
    /* 完全进入核心态 */
```

一组汇编指令恢复软件现场
IRET指令恢复硬件现场
(从核心栈中将相应的值重新装回寄存器中)

```
    /* 恢复现场 */
    RestoreContext();
    /* 手工销毁栈帧 */
    Leave();
    /* 退出中断 */
    InterruptReturn();
}
```

⑥

从核心栈中将保存的值装回CPU内的寄存器

恢复中断前现场

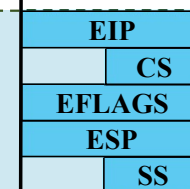
```
#define RestoreContext()
#define Leave()
```

InterruptReturn

```
#define InterruptReturn() \
__asm__ __volatile__ ("iret" :: );
```

核心栈

硬件现场



中断发生时现运行进程在用户态运行



UNIX中断处理流程



(6) : 撤销栈帧

```
void Time::TimeInterruptEntrance()
{
    /* 保存中断现场 */
    SaveContext();
```

```
    /* 完全进入核心态 */
```

一组汇编指令恢复软件现场
IRET指令恢复硬件现场
(从核心栈中将相应的值重新装回寄存器中)

```
    /* 恢复现场 */
    RestoreContext();
    /* 手工销毁栈帧 */
    Leave();
    /* 退出中断 */
    InterruptReturn();
}
```

⑥

恢复中断前现场

```
#define RestoreContext()
#define Leave()
```

InterruptReturn

```
#define InterruptReturn() \
__asm__ __volatile__ ("iret" :: );
```

装回的CS和EIP让进程回到中断前的状态和位置

核心栈

中断发生时现运行进程在用户态运行

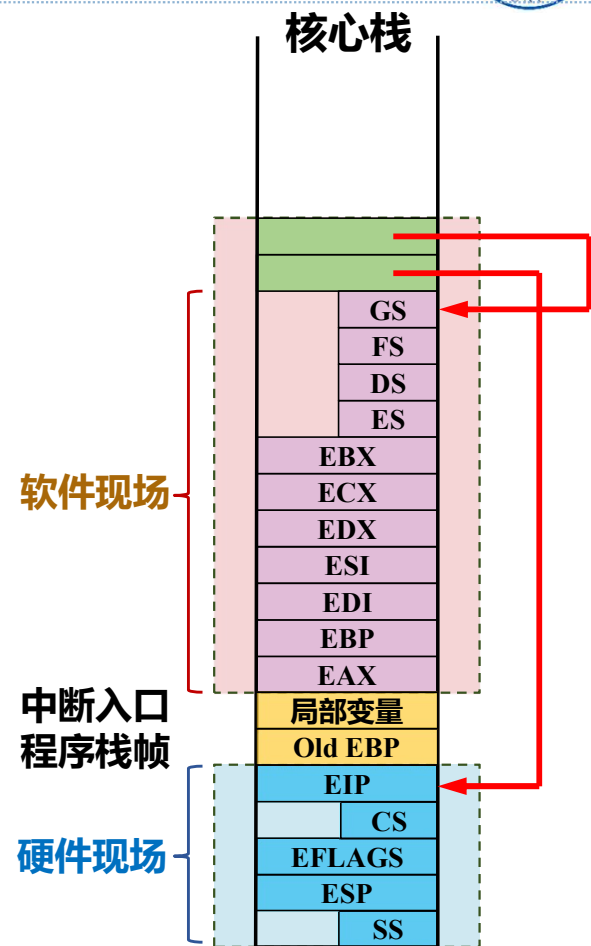
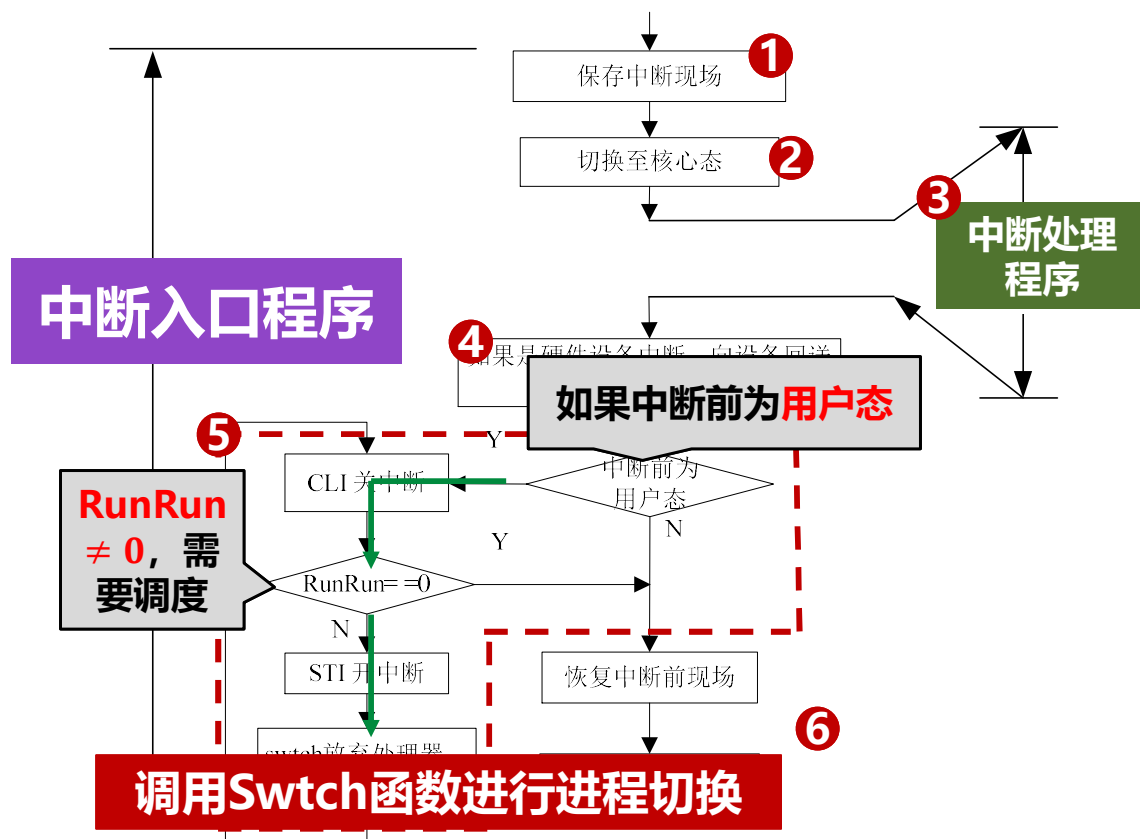


UNIX中断处理流程



(5) : 例行调度

中断入口程序



中断发生时现运行进程在用户态运行

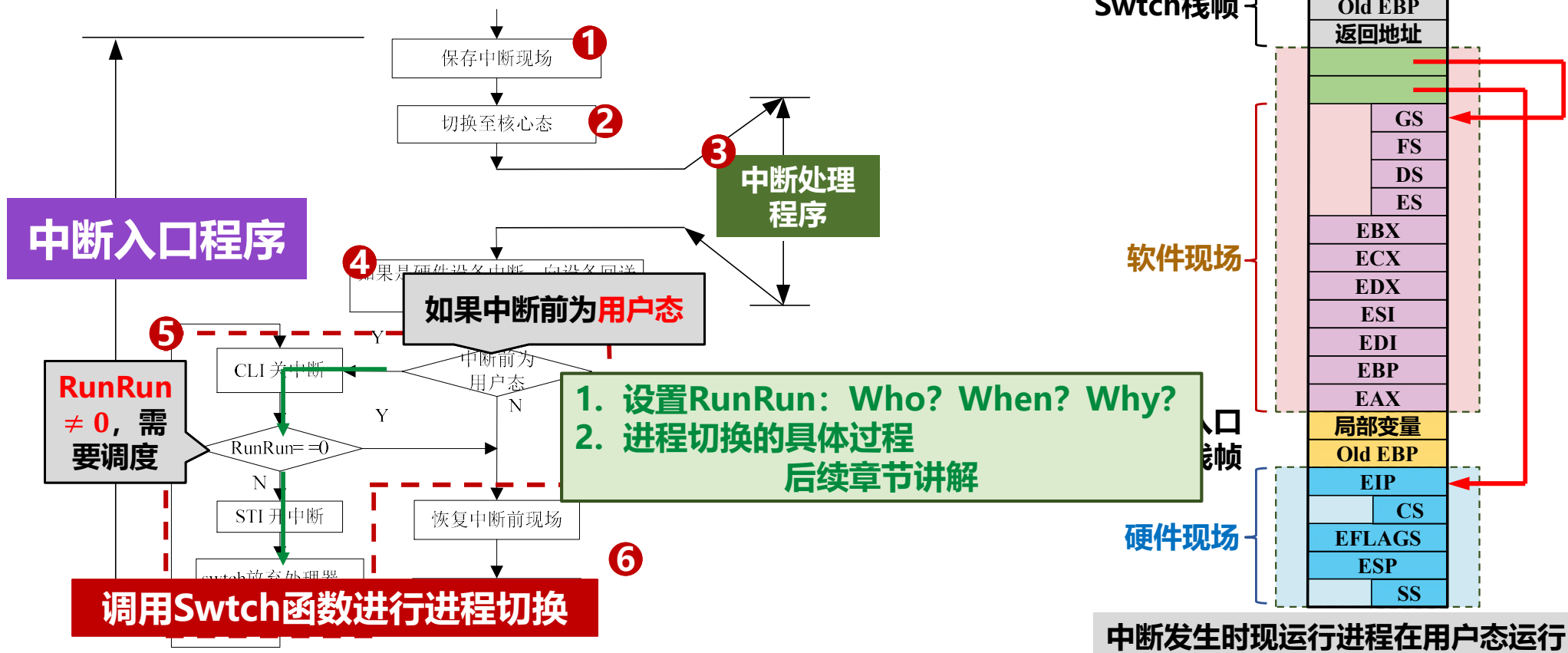


UNIX中断处理流程



(5) : 例行调度

中断入口程序



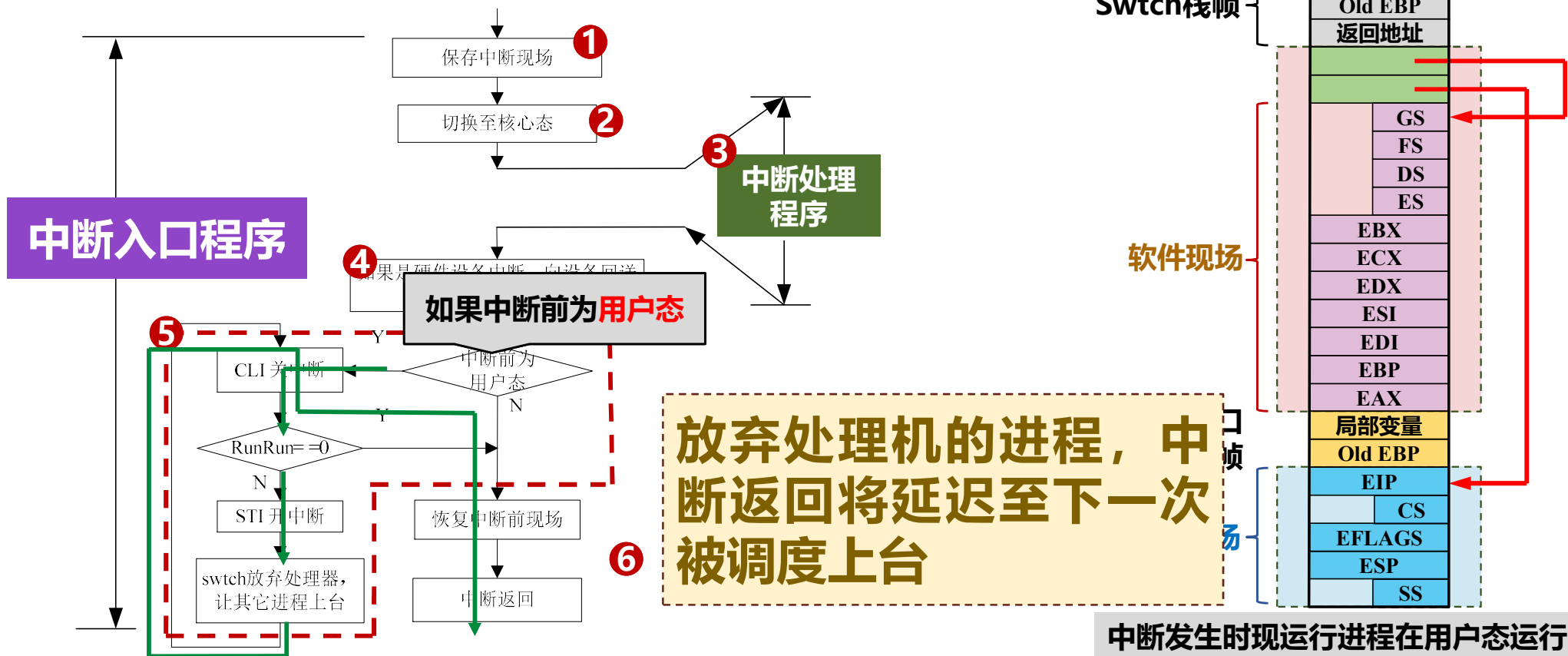


UNIX中断处理流程



(5) : 例行调度

中断入口程序





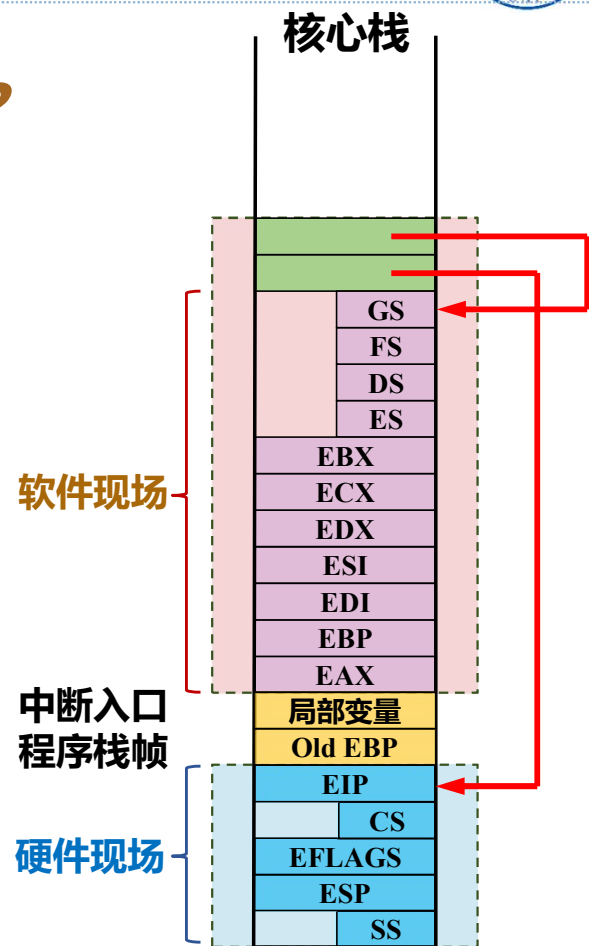
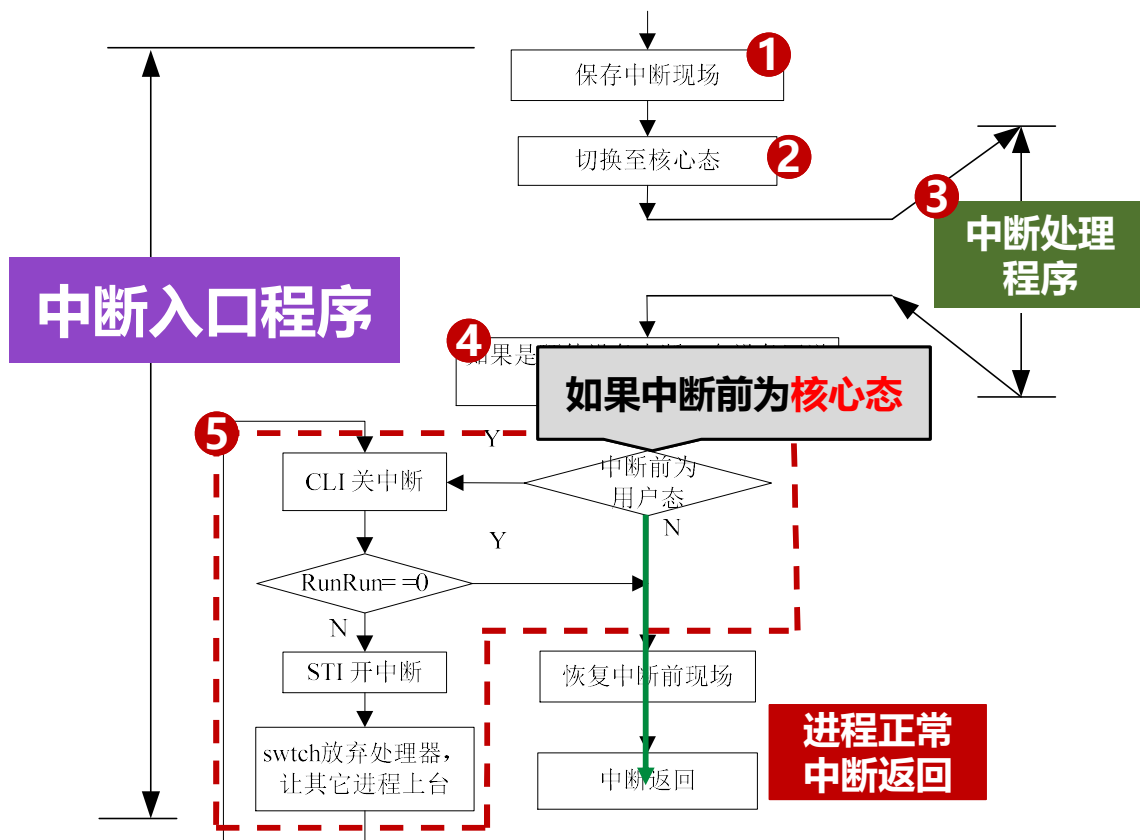
UNIX中断处理流程



(5) : 例行调度

还有其他情况吗?

中断入口程序



中断发生时现运行进程在用户态运行

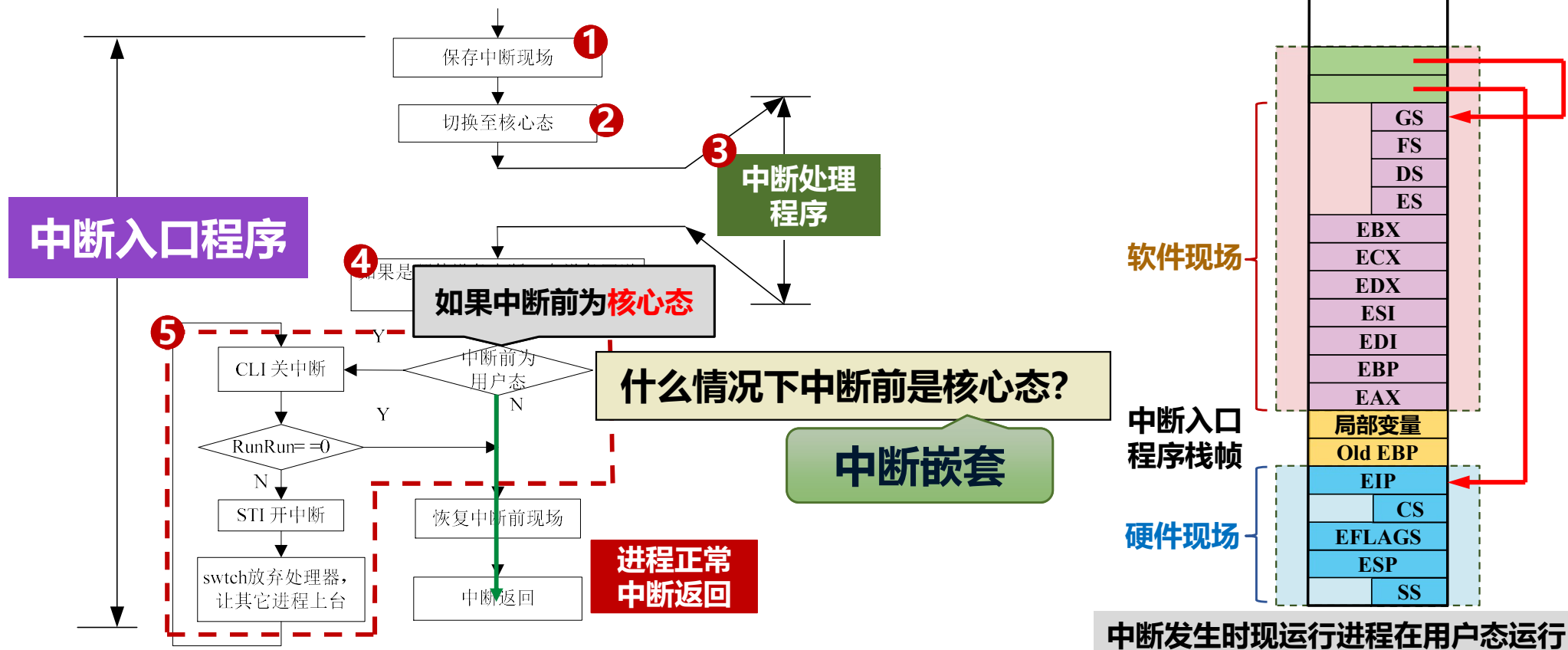


UNIX中断处理流程



(5) : 例行调度

中断入口程序



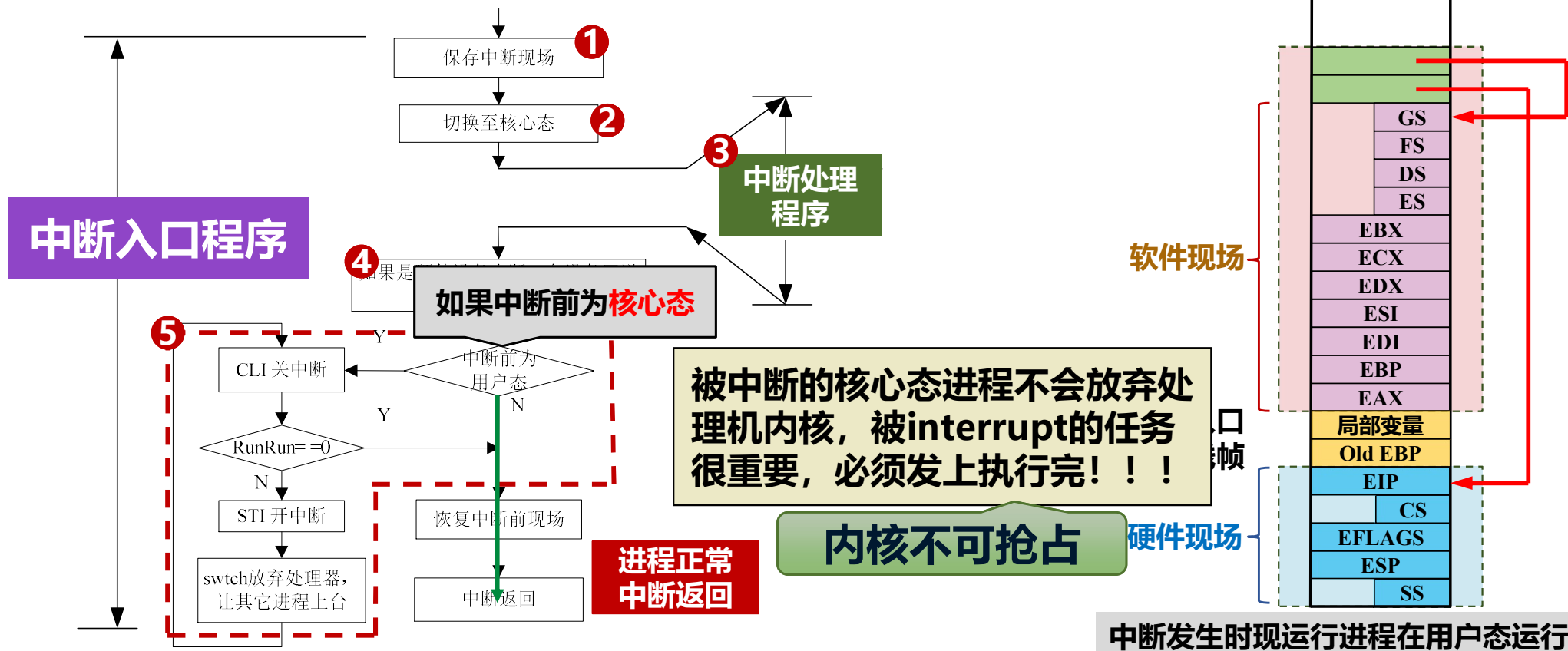


UNIX中断处理流程



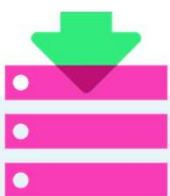
(5) : 例行调度

中断入口程序





你必须知道的关于中断的细节



栈帧的变化



进程的切换



中断的地位



中断的扩展



UNIX中断处理流程



中断处理过程中栈帧的变化

中断响应前.....

核心栈

高地址

中断发生时现运行进程在用户态运行



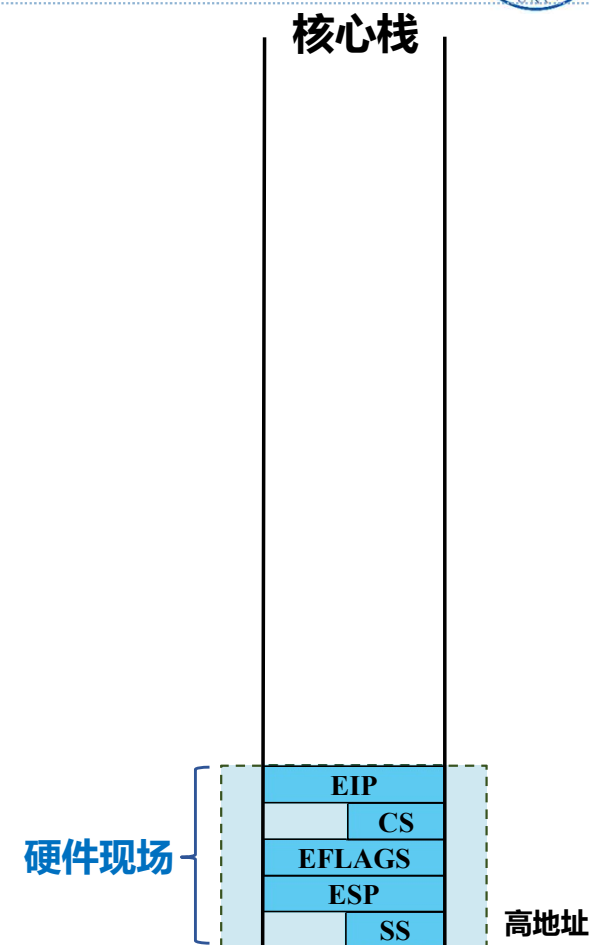
UNIX中断处理流程



中断处理过程中栈帧的变化

中断隐指令执行结束时.....

中断隐指令 ●



中断发生时现运行进程在用户态运行



UNIX中断处理流程



跳转到中断入口程序时.....

中断隐指令

硬件调用C语言程序

中断入口程序

内联的汇编指令

C语言

内联的汇编指令

```
void Time::TimeInterruptEntrance()
{
    /* 保存中断现场 */
    SaveContext();

    /* 完全进入核心态 */
    SwitchToKernel();

    /* 调用时钟中断处理程序 */
    CallHandler(Time, Clock);

    /* 例行调度 */
    .....

    /* 恢复现场 */
    RestoreContext();

    /* 手工销毁栈帧 */
    Leave();

    /* 退出中断 */
    InterruptReturn();
}
```

核心栈

中断入口
程序栈帧

硬件现场

局部变量

Old EBP

EIP

CS

EFLAGS

ESP

SS

指向用户栈

高地址

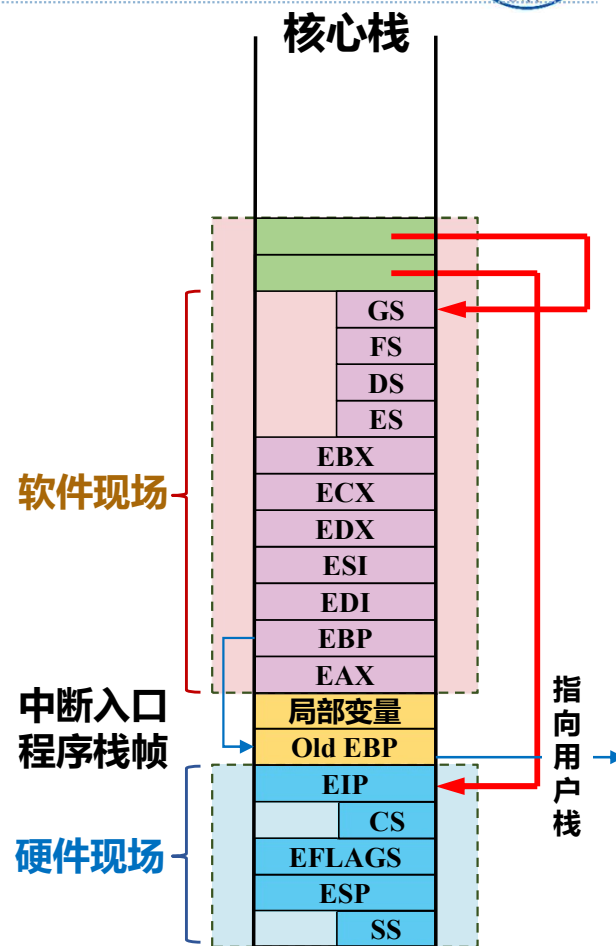
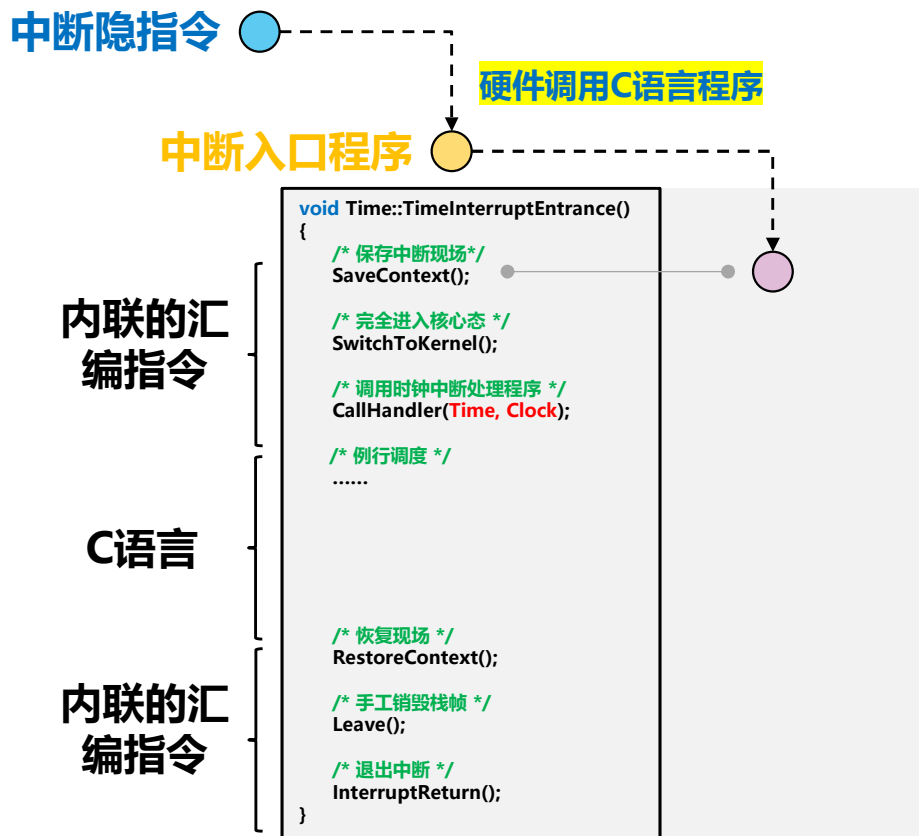
中断发生时现运行进程在用户态运行



UNIX中断处理流程



中断入口程序中完成软件现场保护时.....



中断发生时现运行进程在用户态运行

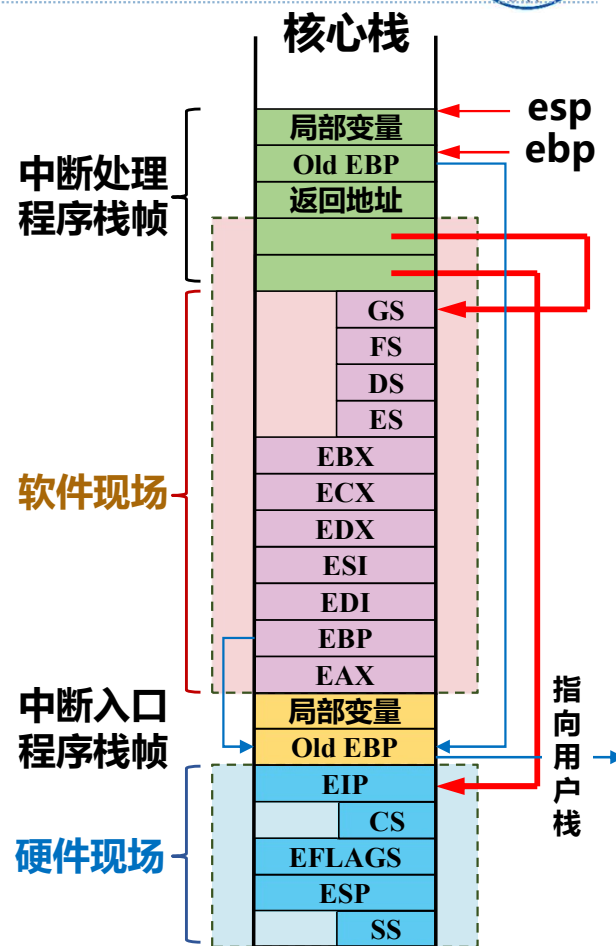
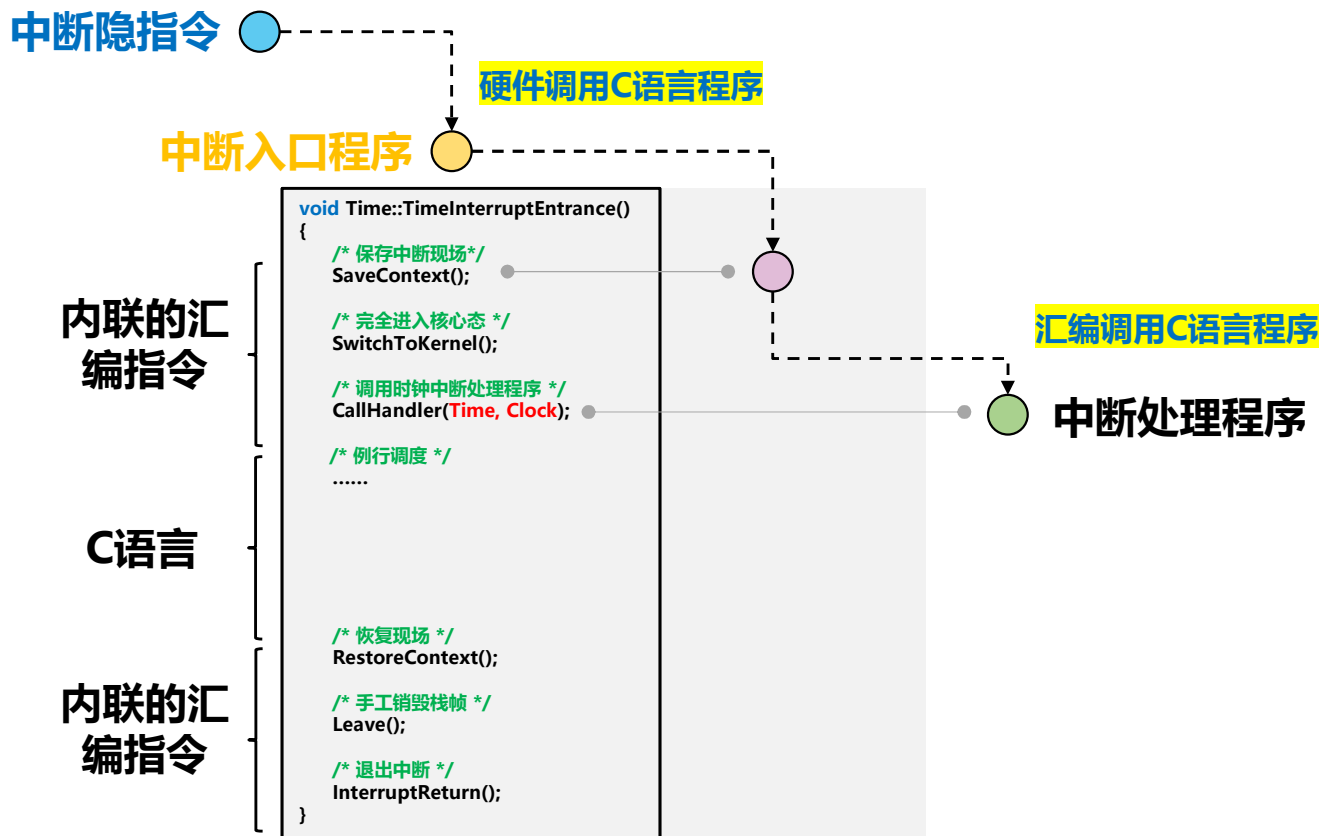


UNIX中断处理流程



中断处理过程中栈帧的变化

跳转到中断处理程序时.....



中断发生时现运行进程在用户态运行

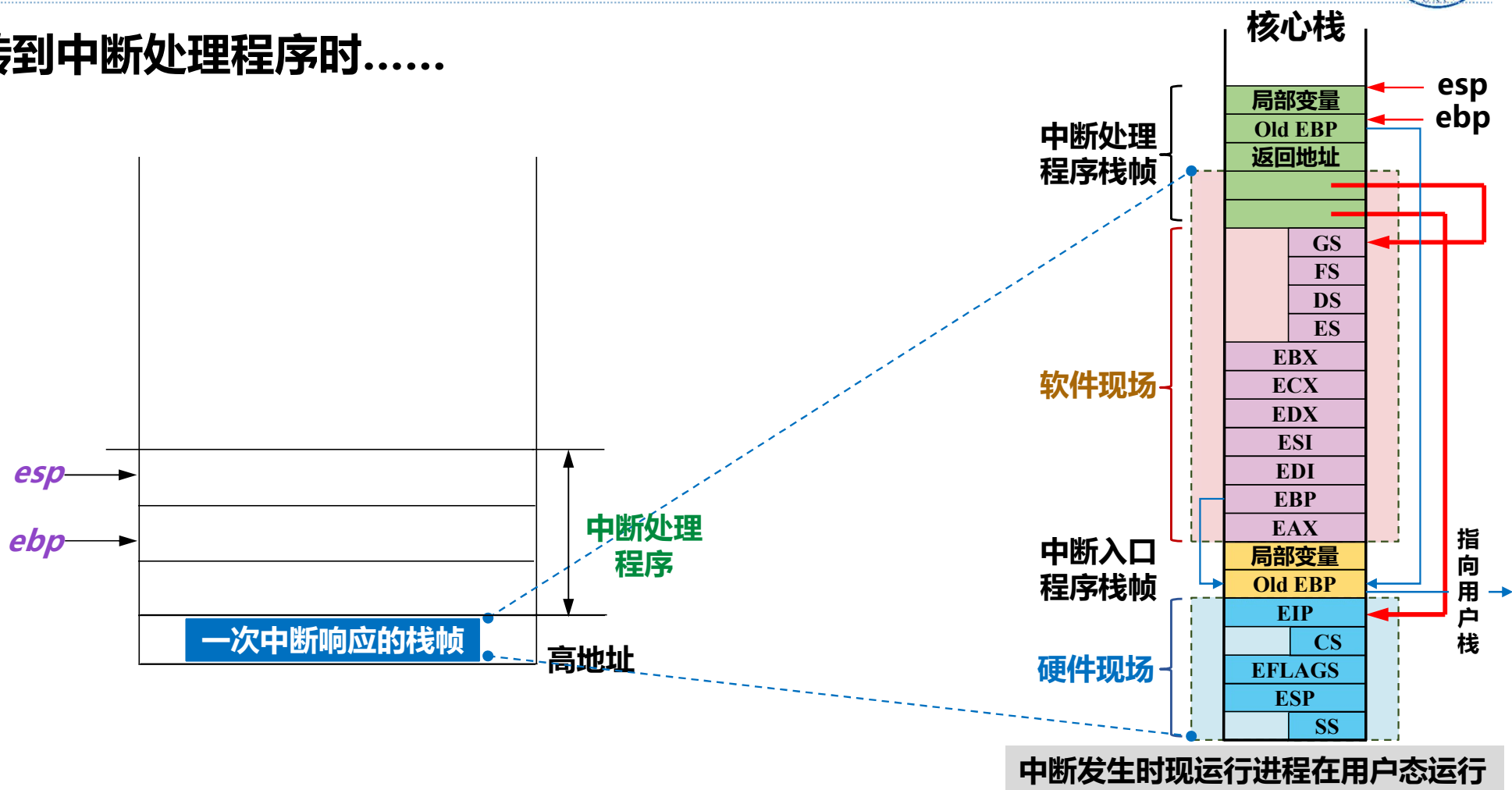


UNIX中断处理流程



中断处理过程中栈帧的变化

跳转到中断处理程序时.....



中断发生时现运行进程在用户态运行

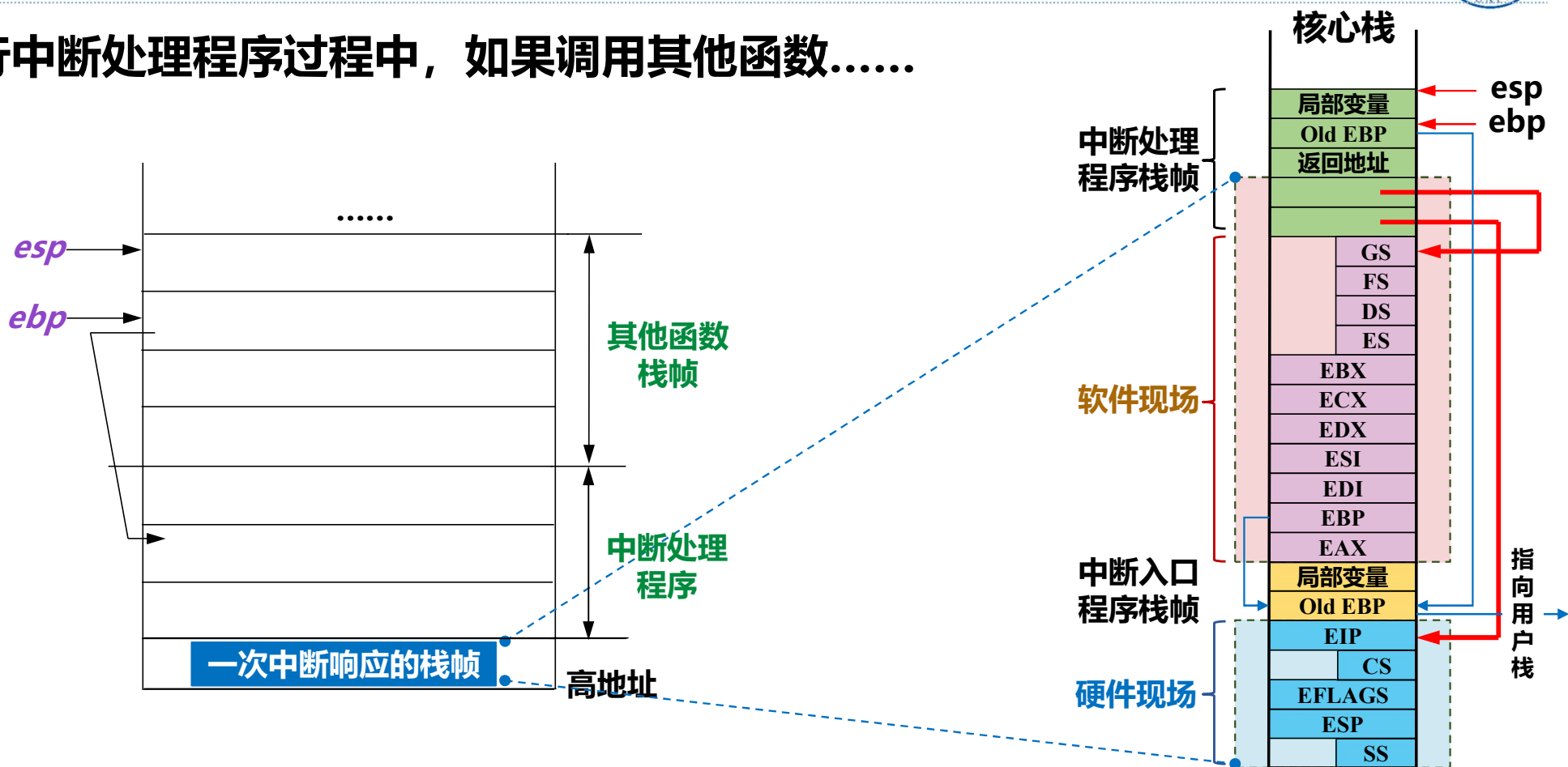


UNIX中断处理流程



中断处理过程中栈帧的变化

执行中断处理程序过程中，如果调用其他函数.....



中断发生时现运行进程在用户态运行

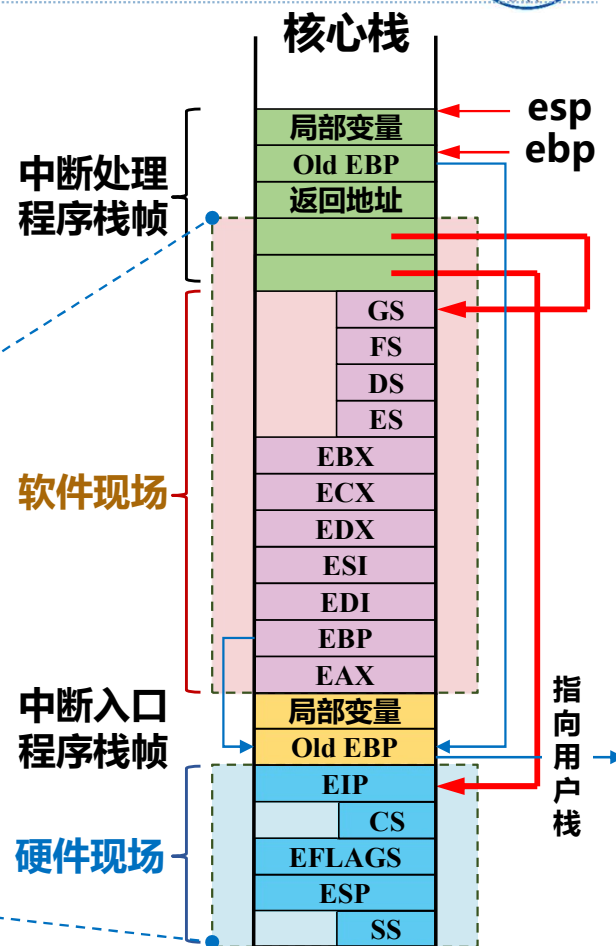
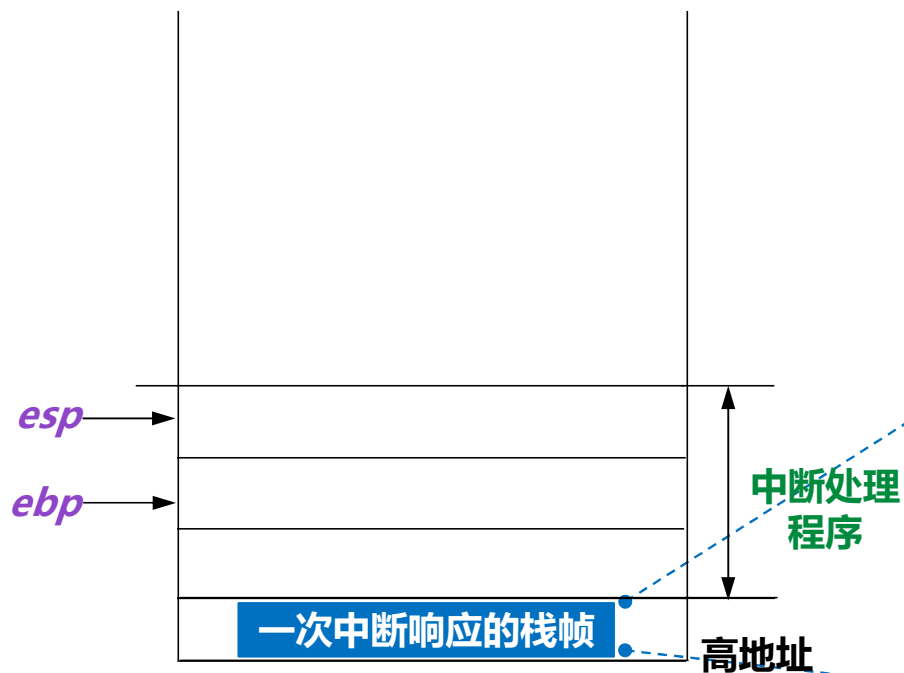


UNIX中断处理流程



中断处理过程中栈帧的变化

返回中断处理程序后.....



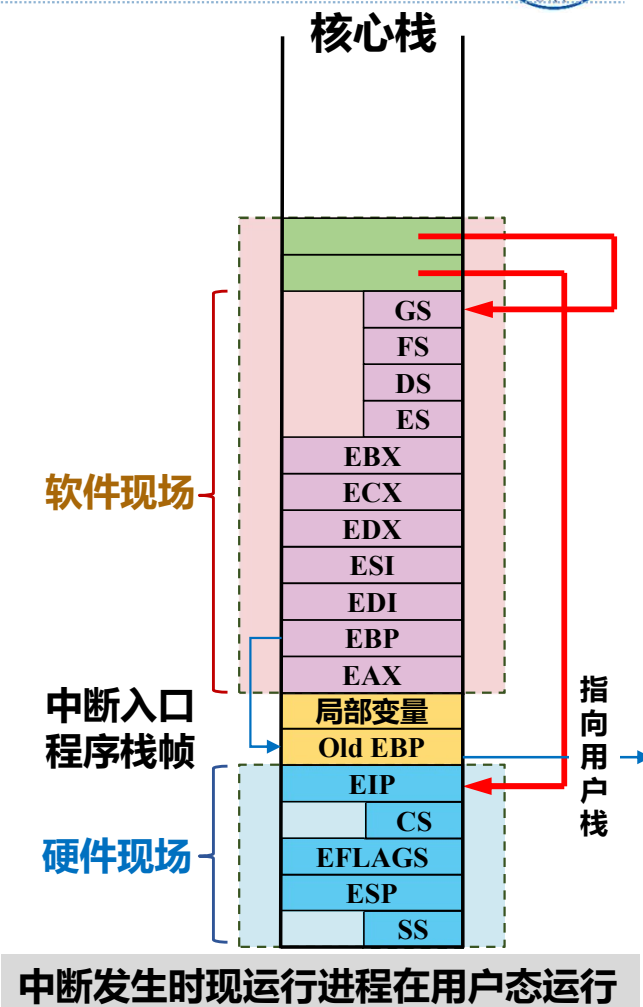
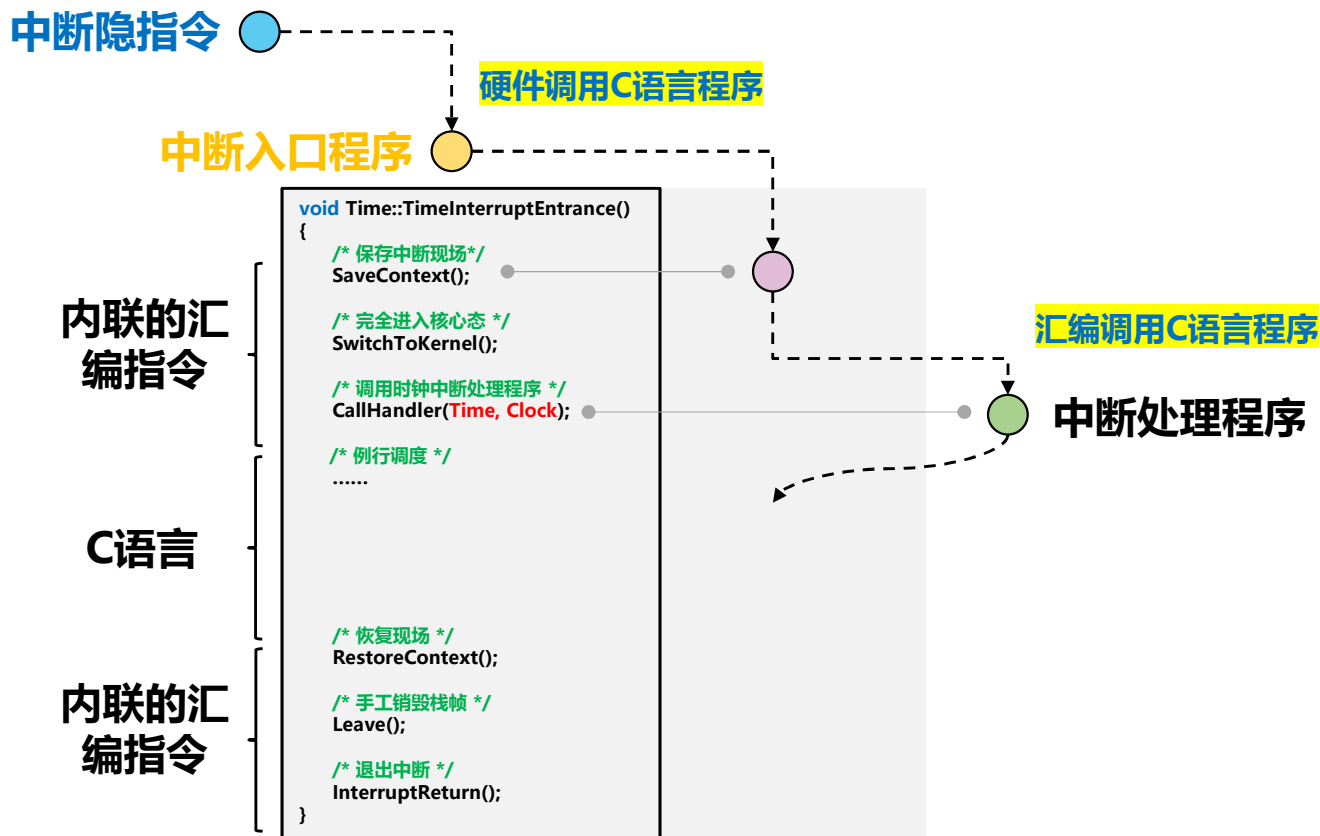
中断发生时现运行进程在用户态运行



UNIX中断处理流程



返回中断入口程序后.....

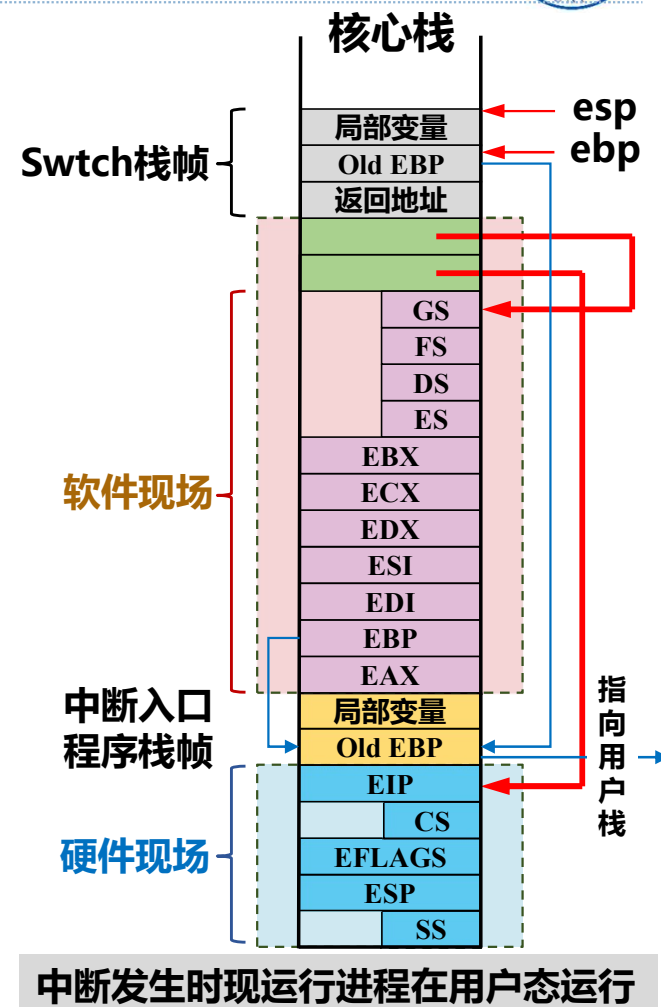
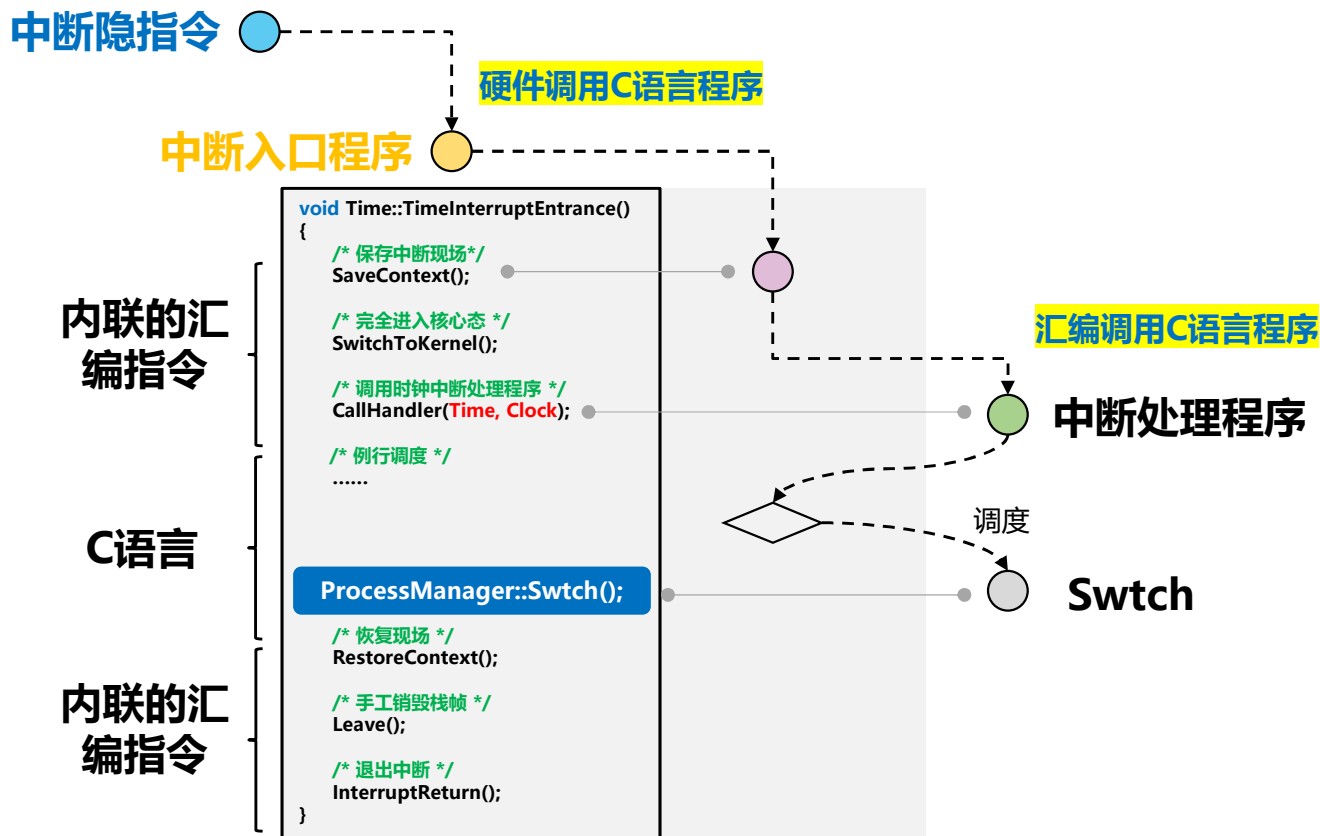




UNIX中断处理流程



返回中断入口程序后，如果调度.....



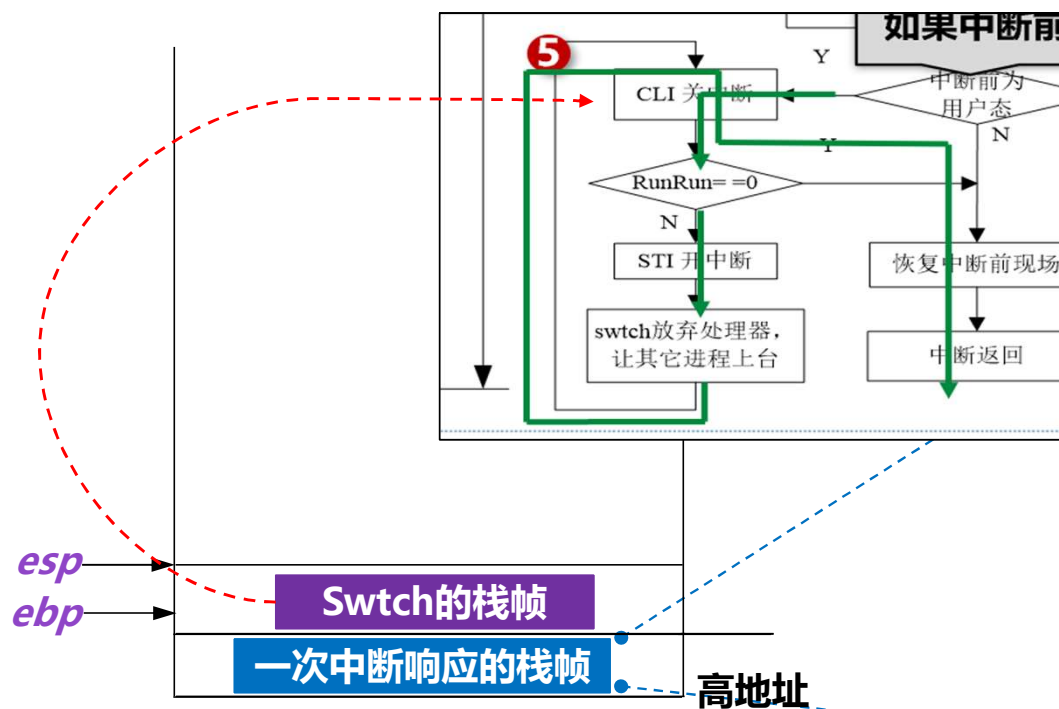


UNIX中断处理流程

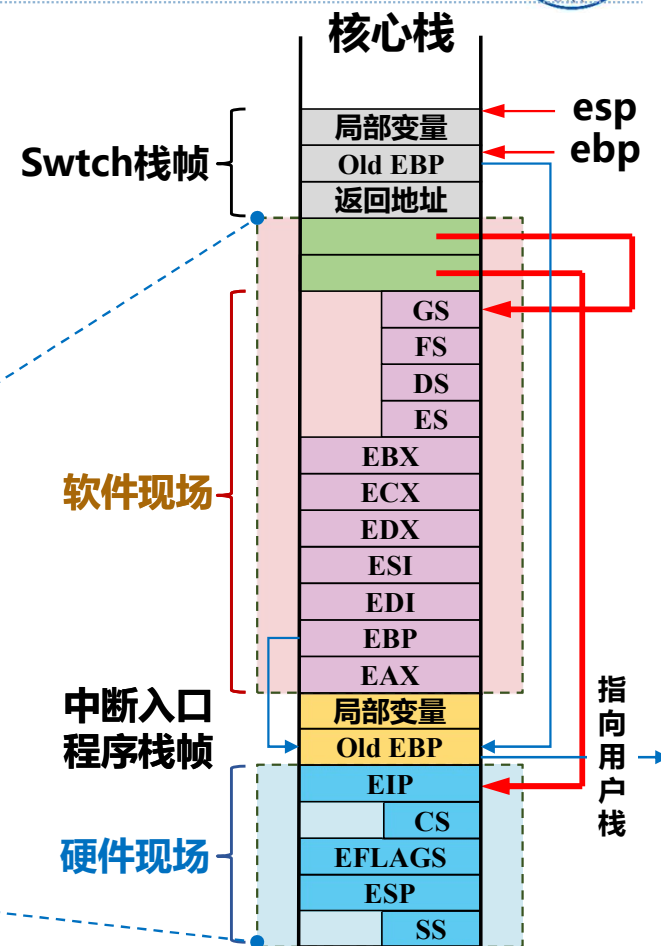


中断处理过程中栈帧的变化

返回中断入口程序后，如果调度.....



Swtch栈帧里记录的返回地址?



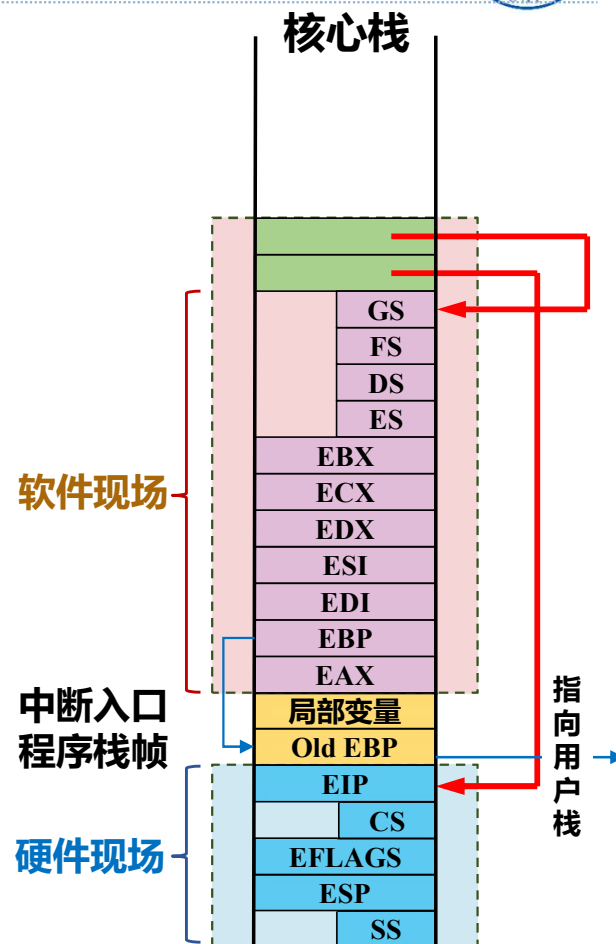
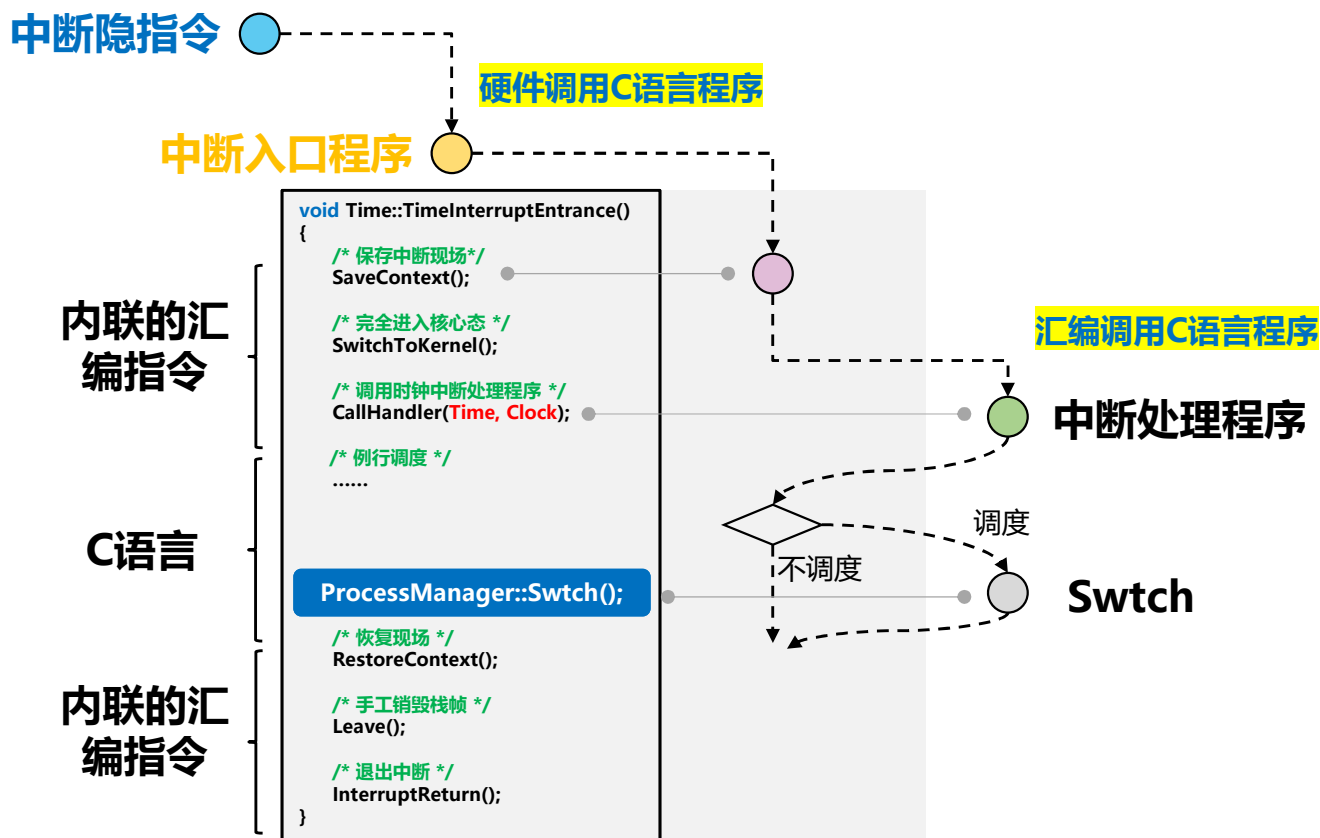
中断发生时现运行进程在用户态运行



UNIX中断处理流程



返回中断入口程序后，如果不调度或者调度返回.....



中断发生时现运行进程在用户态运行

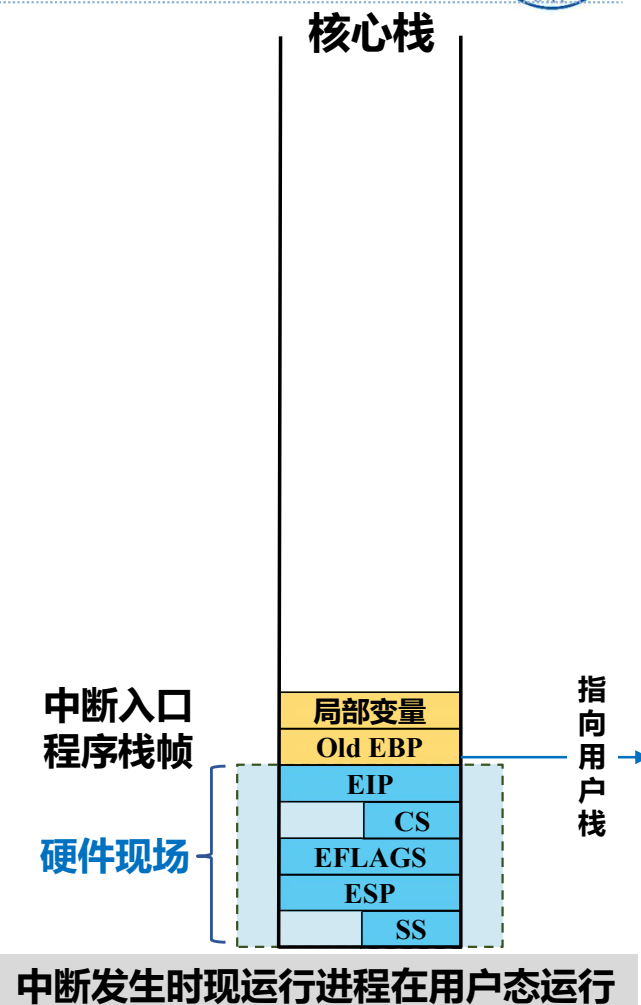
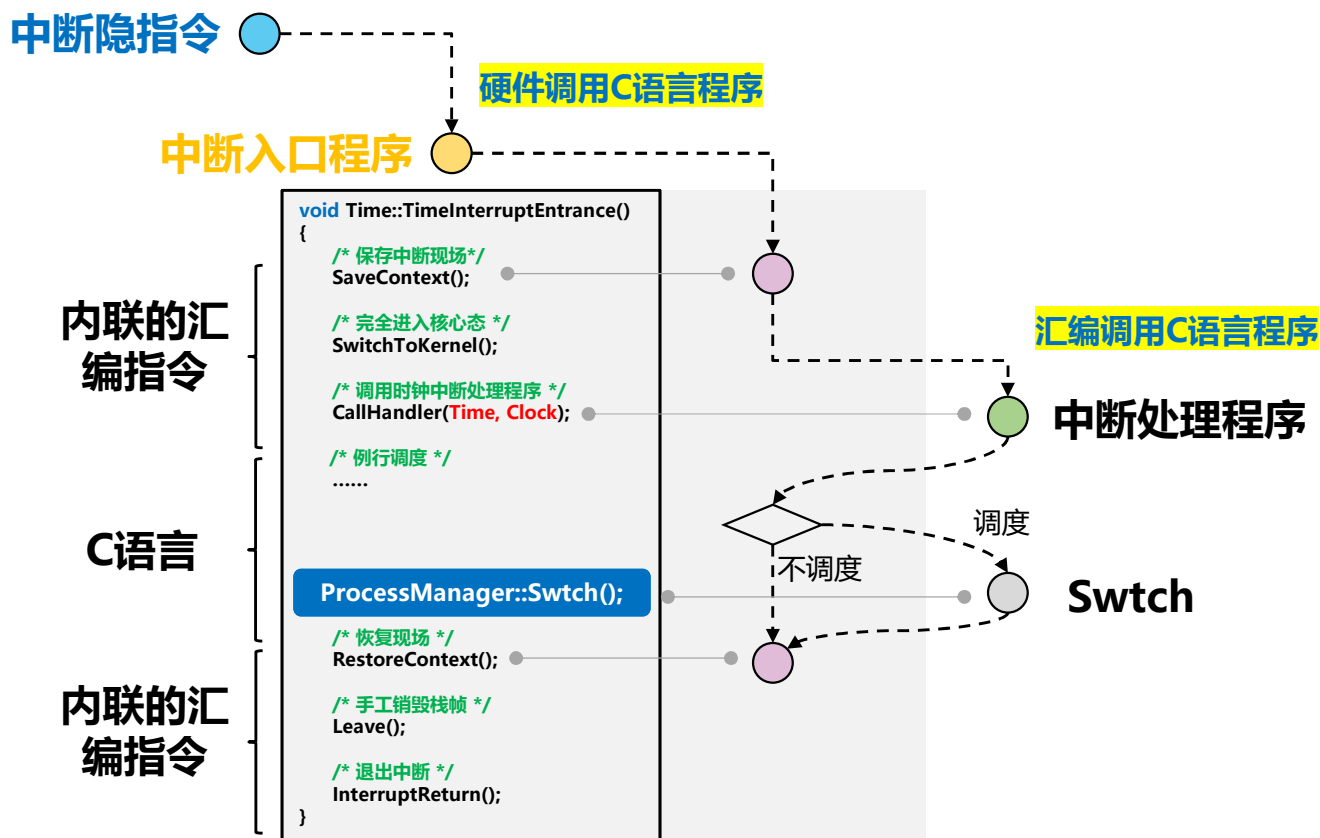


UNIX中断处理流程



中断处理过程中栈帧的变化

返回中断入口程序后，如果不调度或者调度返回.....

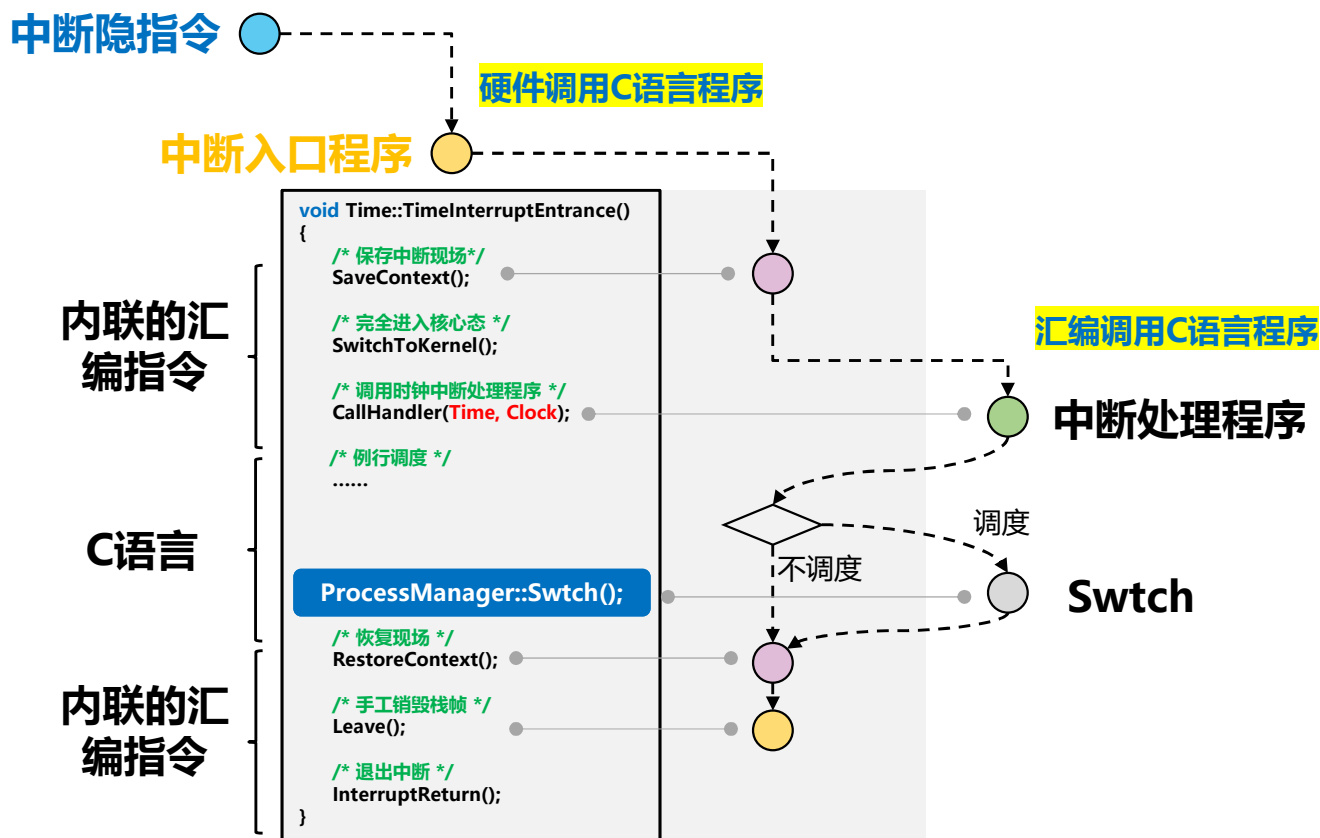




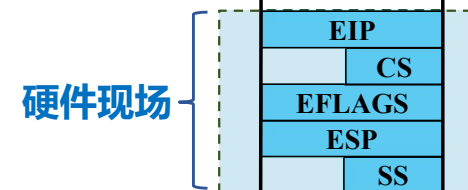
UNIX中断处理流程



返回中断入口程序后，如果不调度或者调度返回.....



核心栈



中断发生时现运行进程在用户态运行

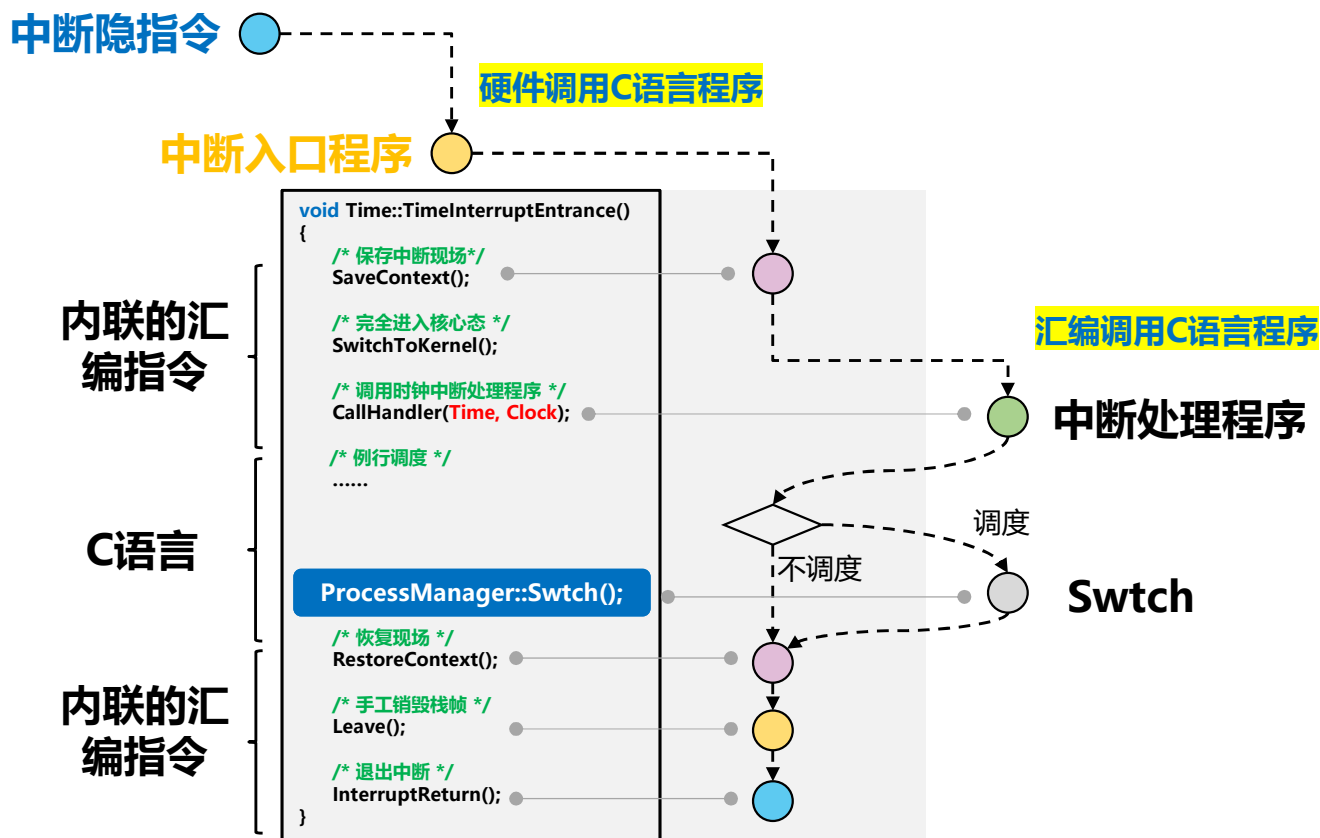


UNIX中断处理流程



返回中断入口程序后，如果不调度或者调度返回.....

核心栈



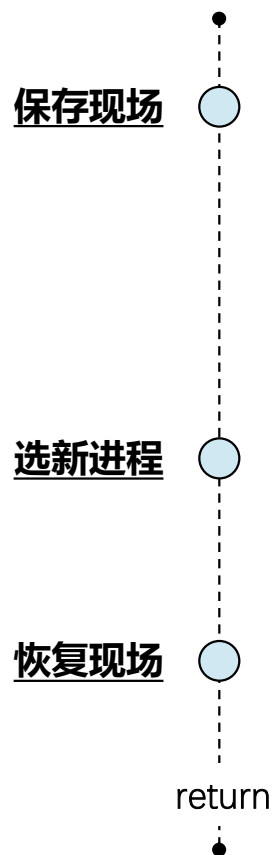
中断发生时现运行进程在用户态运行



UNIX中断处理流程



ProcessManager::Swch

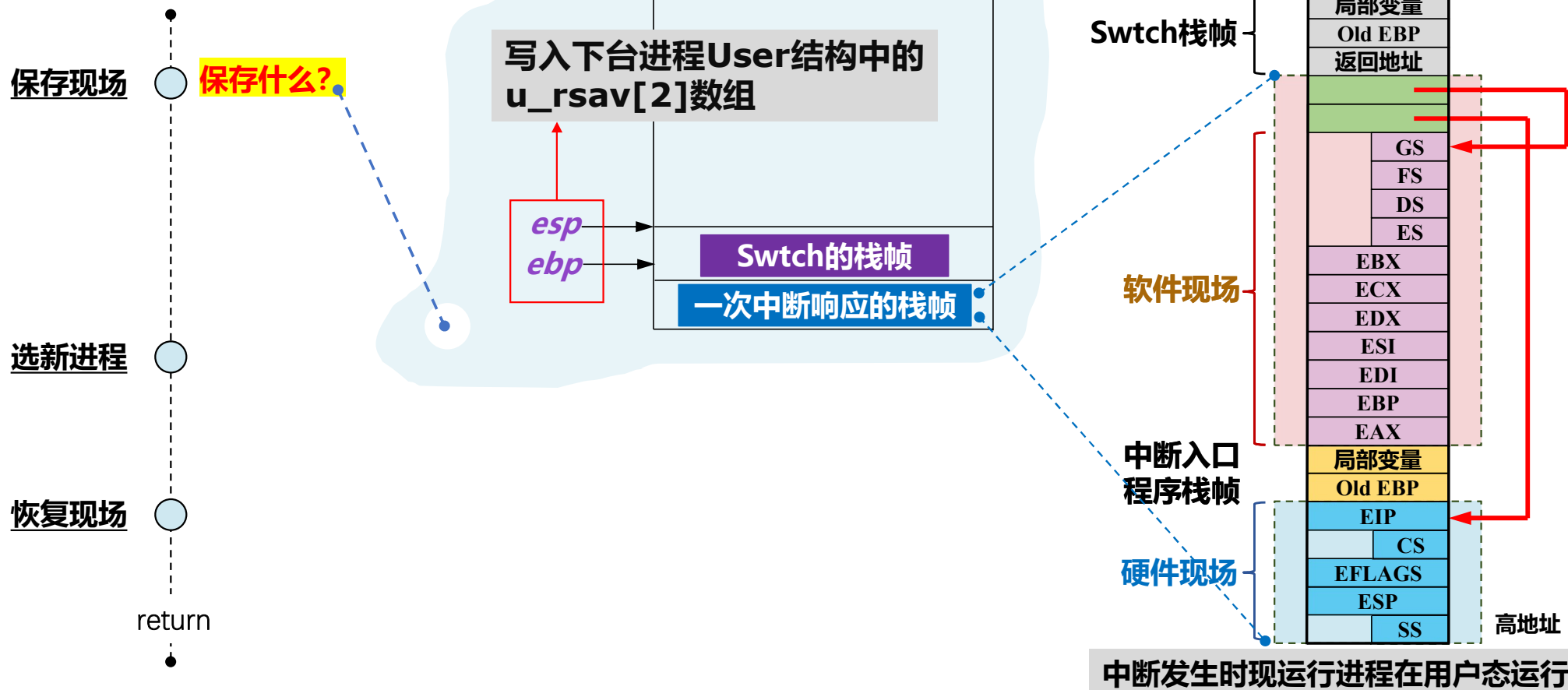




UNIX中断处理流程



ProcessManager::Swch

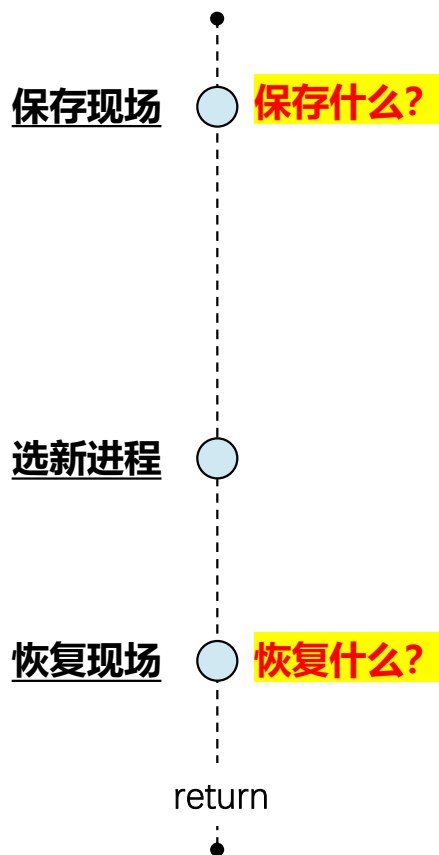




UNIX中断处理流程



ProcessManager::Swch

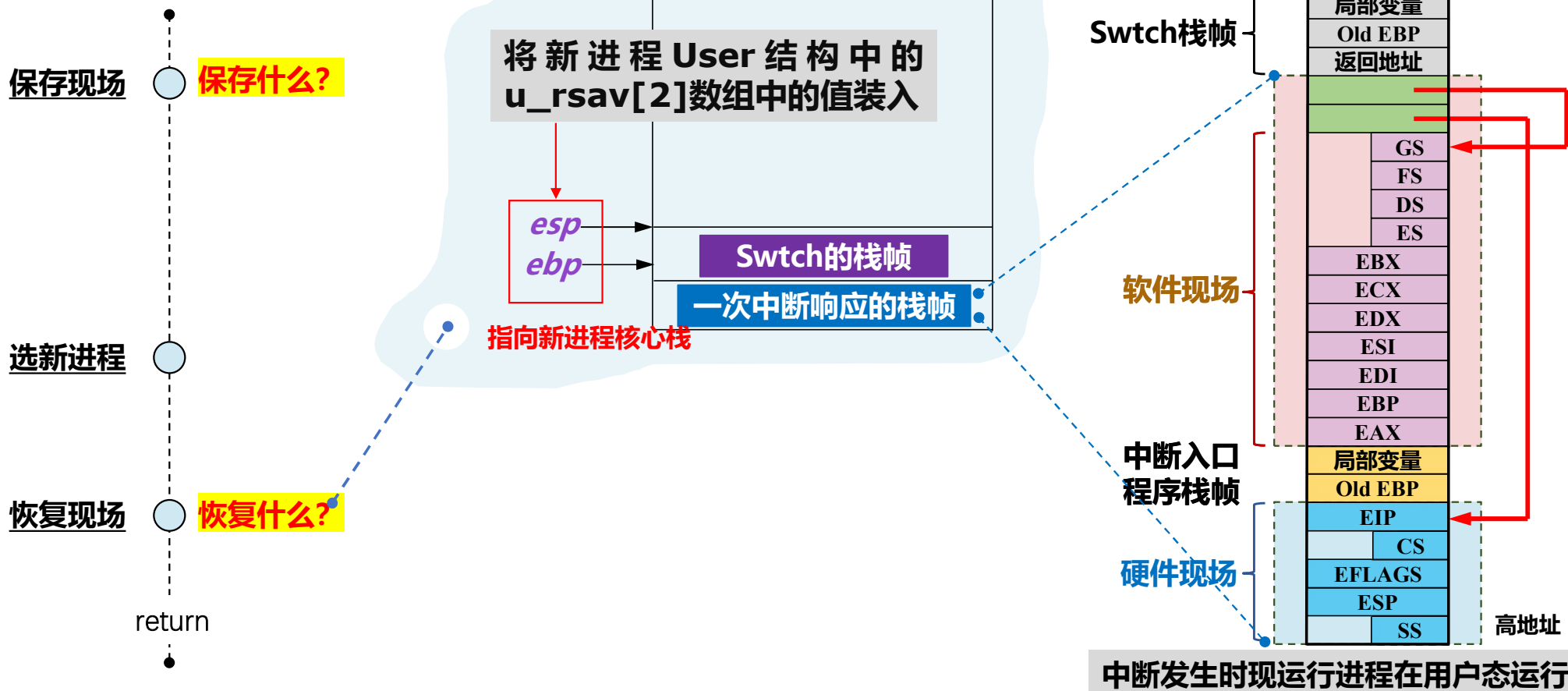




UNIX中断处理流程



ProcessManager::Swch

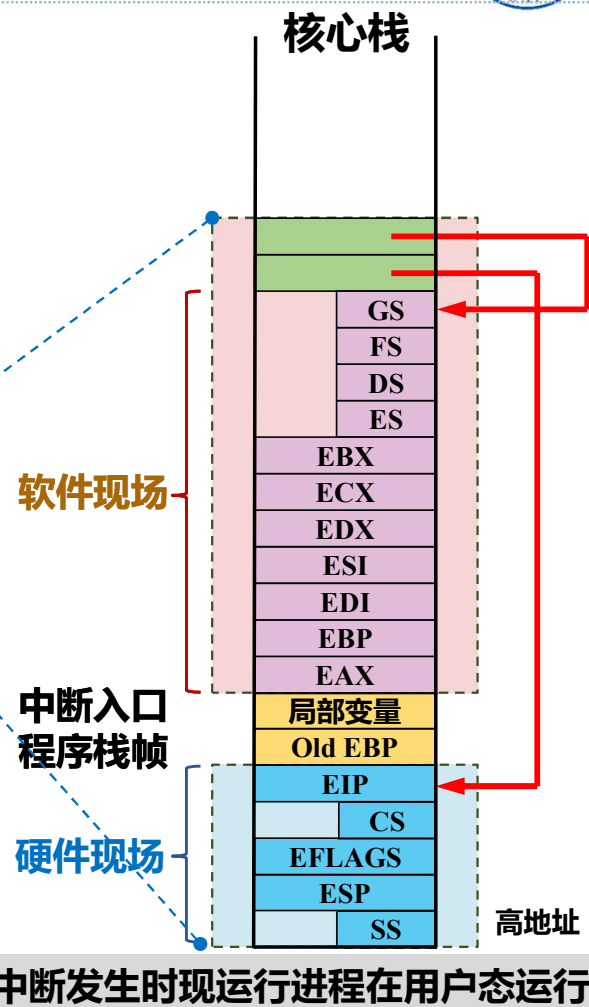
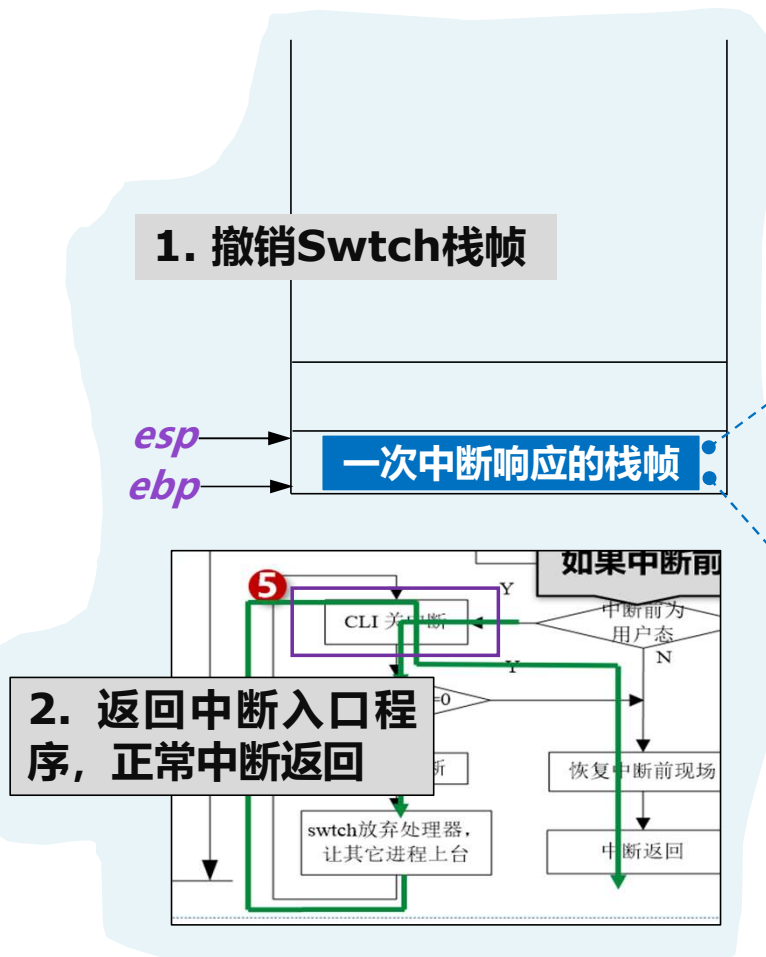
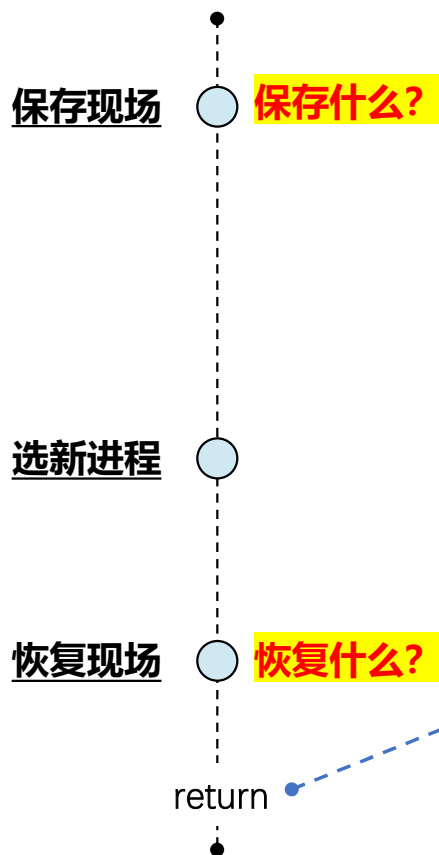




UNIX中断处理流程



ProcessManager::Swch



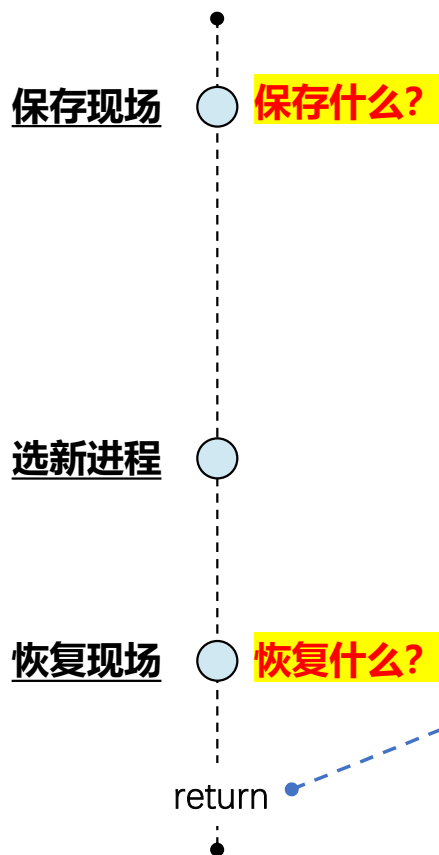
中断发生时现运行进程在用户态运行



UNIX中断处理流程



ProcessManager::Swth



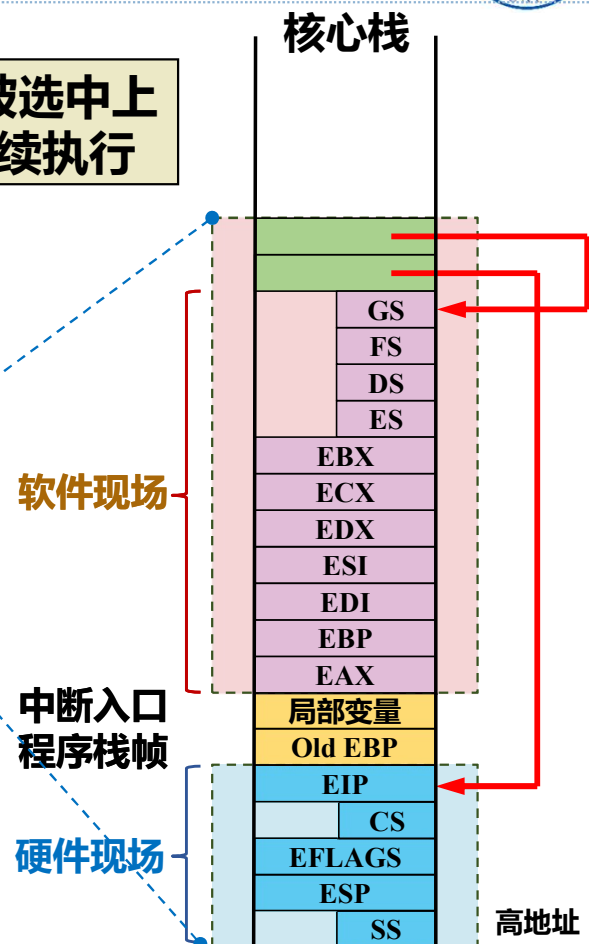
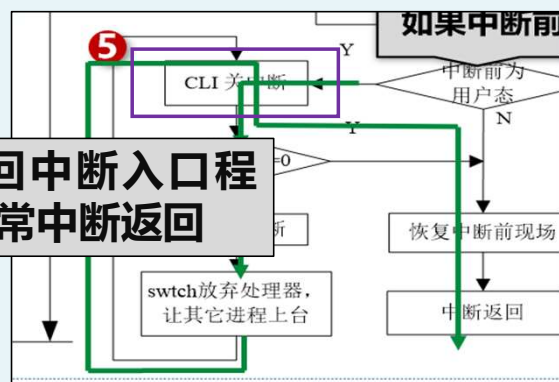
所有中断返回前下台的进程，只有再次被选中上台，才会从Swth回到中断入口程序继续执行

1. 撤销Swth栈帧

esp
ebp

一次中断响应的栈帧

2. 返回中断入口程序，正常中断返回



中断发生时现运行进程在用户态运行



中断的基本概念

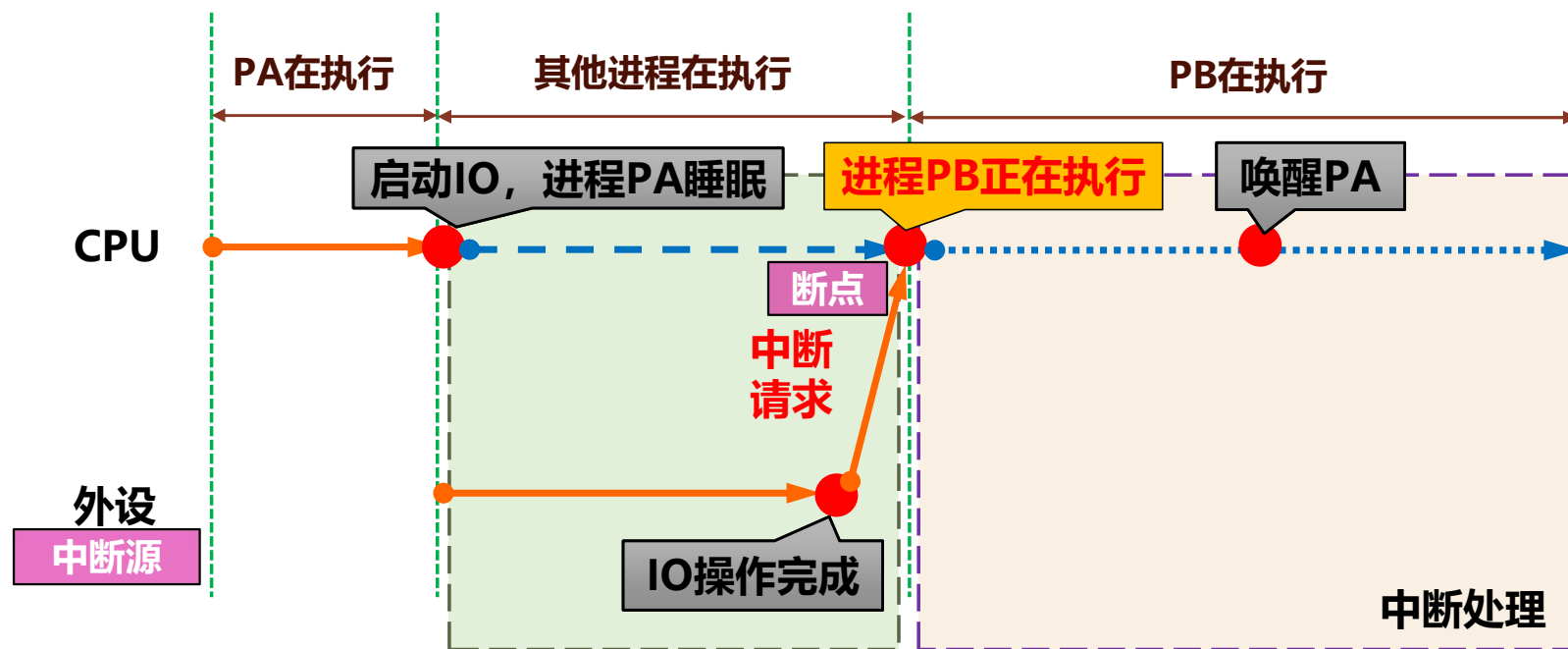
CPU不可能时刻查询外设的工作状态。。。



什么是中断

一种设备控制方式

一种外设的数据传输方式



1. 保证了CPU和设备之间的并行操作

2. 提供了进程执行内核代码的机会



中断的基本概念

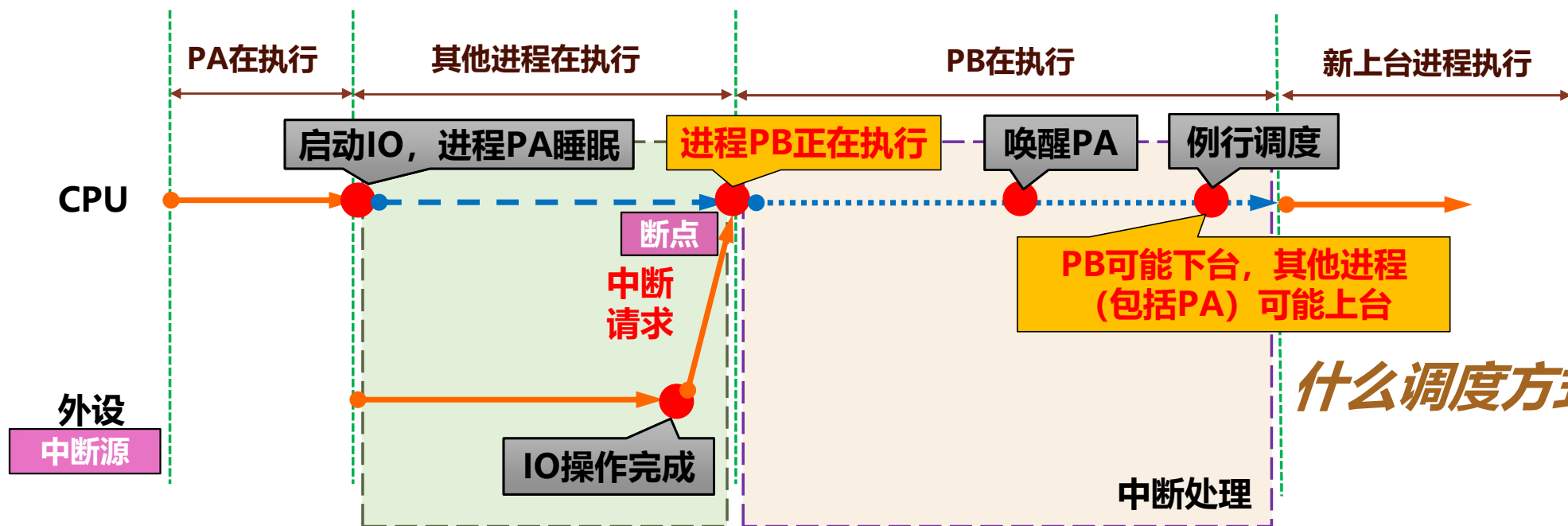
CPU不可能时刻查询外设的工作状态。。。



什么是中断

一种设备控制方式

一种外设的数据传输方式



什么调度方式?

1. 保证了CPU和设备之间的并行操作

2. 提供了进程执行内核代码的机会



中断的基本概念

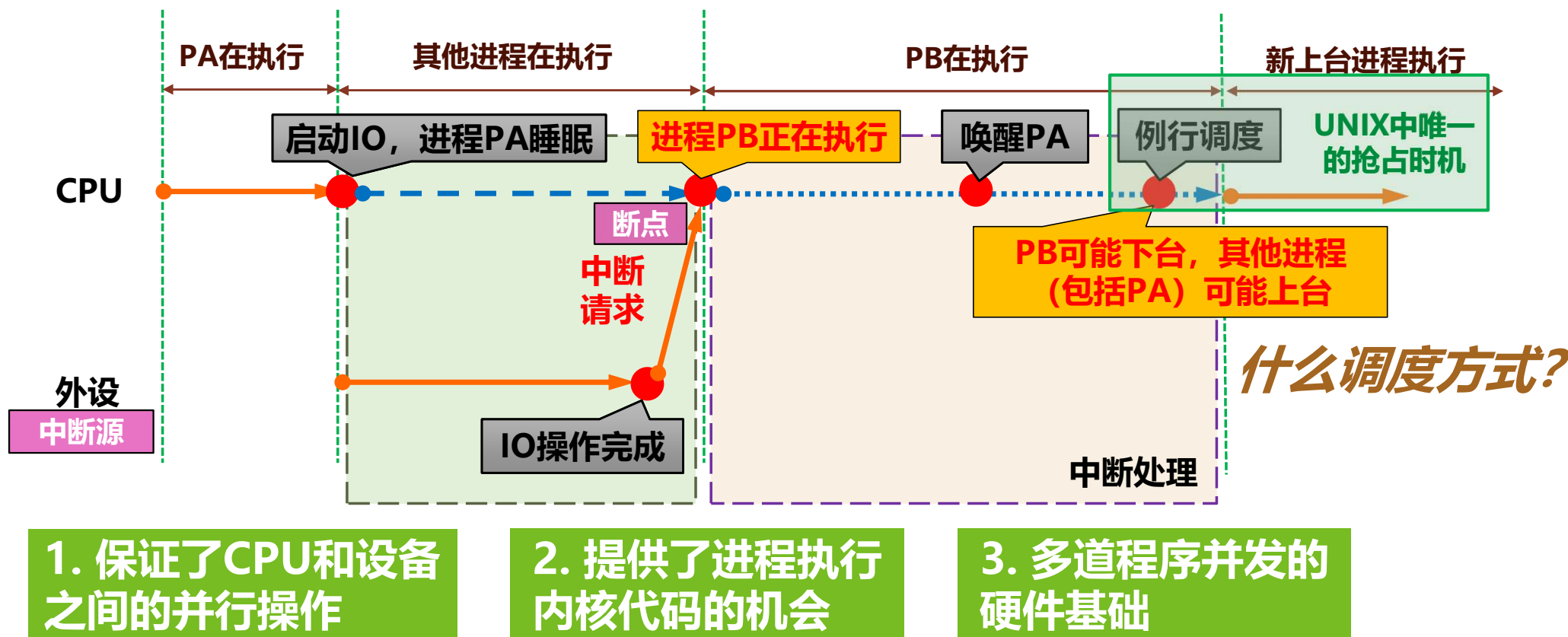
CPU不可能时刻查询外设的工作状态。。。



什么是中断

一种设备控制方式

一种外设的数据传输方式





UNIX中断处理流程



如何看待中断



站在进程PA的角度：

执行

我要用外设

CPU太快不等我

阻塞

先把CPU让给别人

中断处理中我被叫醒

就绪

等下一次上台的机会



站在进程PB的角度：

我正在执行自己的程序

中断来了

被迫执行内核代码

叫醒一个和我无关的进程

中断结束可能被剥夺CPU

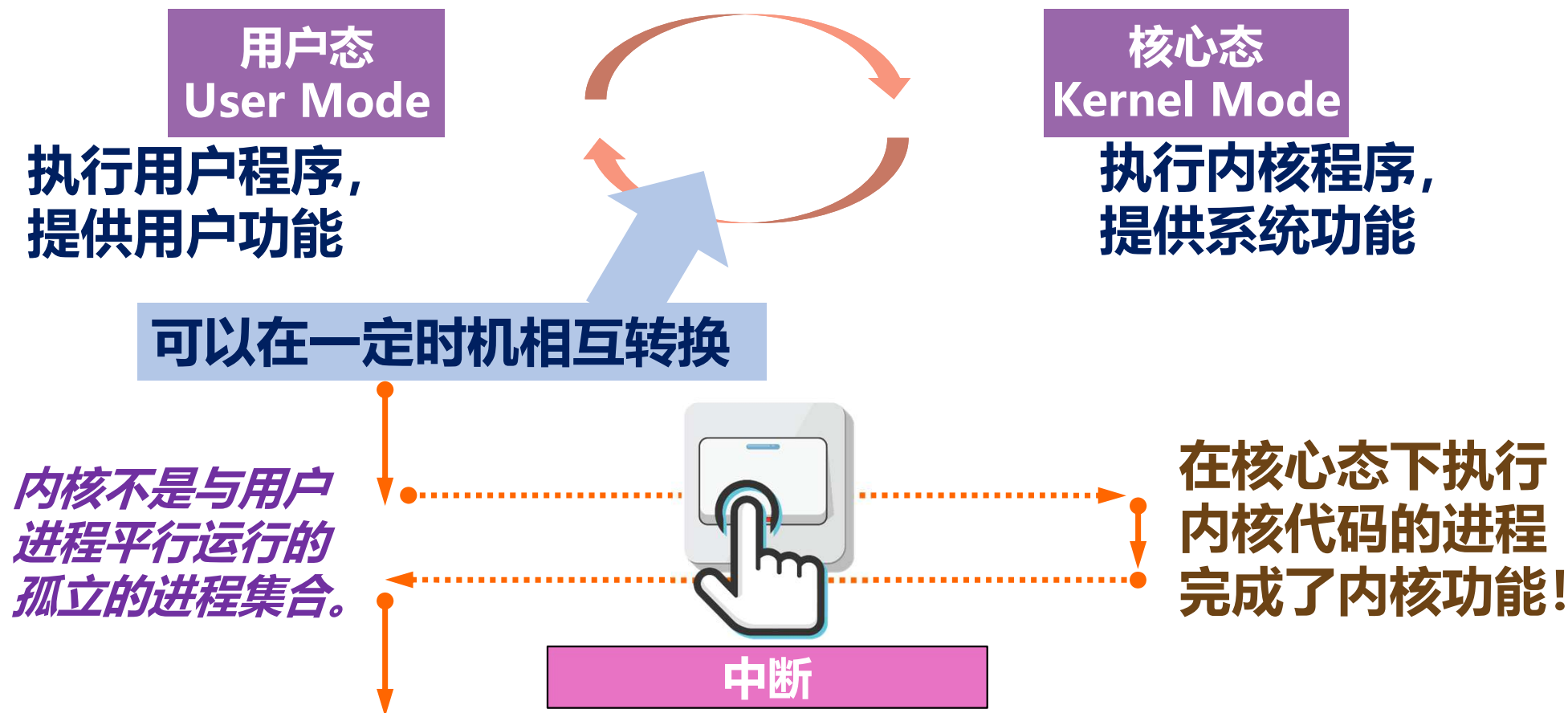
执行

被选为班长

就绪



UNIX中进程的两种执行状态

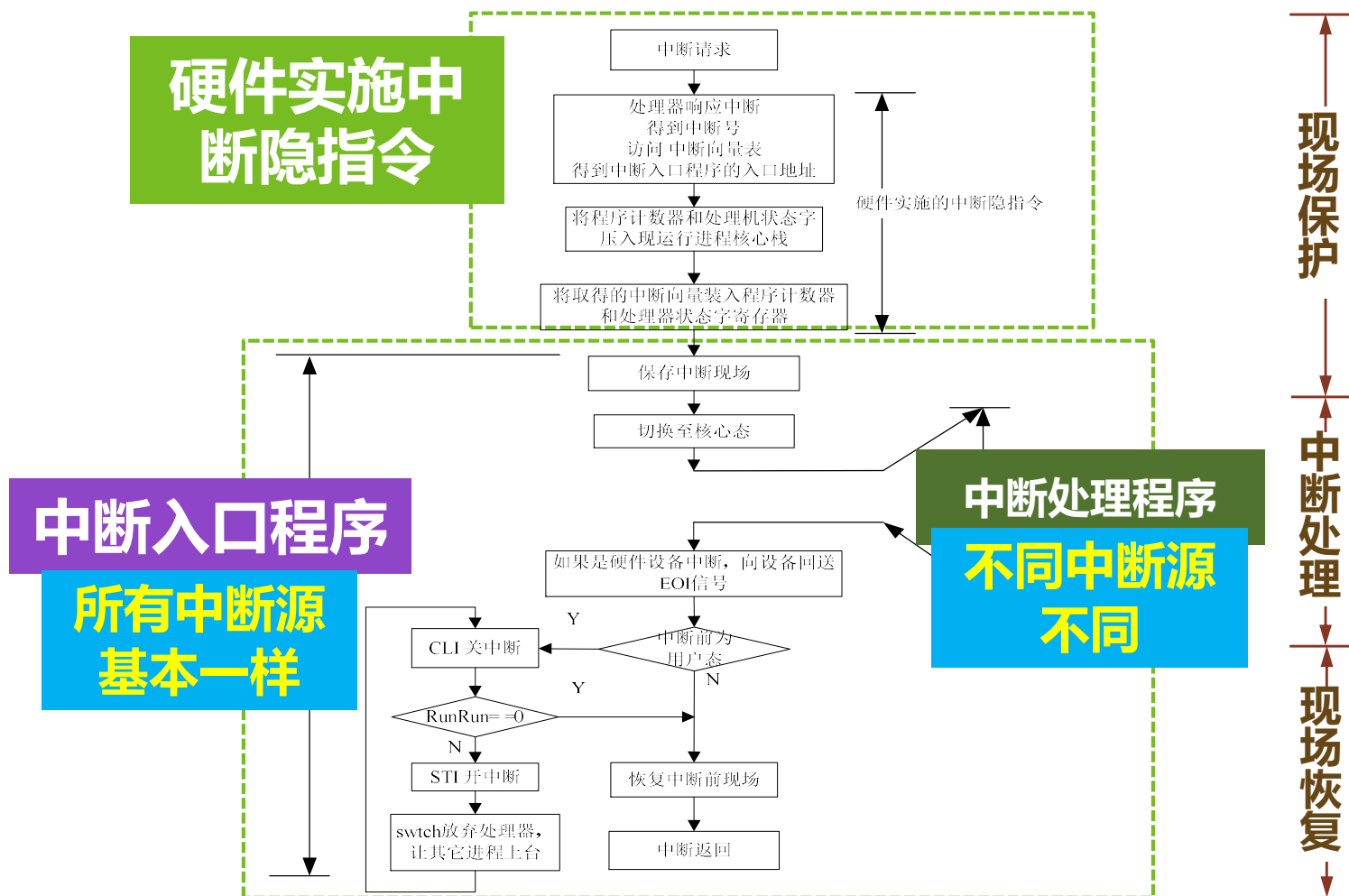




UNIX中断处理流程



如何看待中断



回看整个中断处理的过程。。



如果把中断关掉会发生什么？



现代计算机系统中，中断的概念被扩展.....

中断源

CPU外

1. I/O完成或出错引起的设备中断

中断

CPU停下正在执行的代码，转去执行一段特定的内核代码

CPU内

2. 断点跟踪

3. 硬件故障（奇、偶校验错、电源）

4. 程序性故障（浮点、除0、地址溢出）

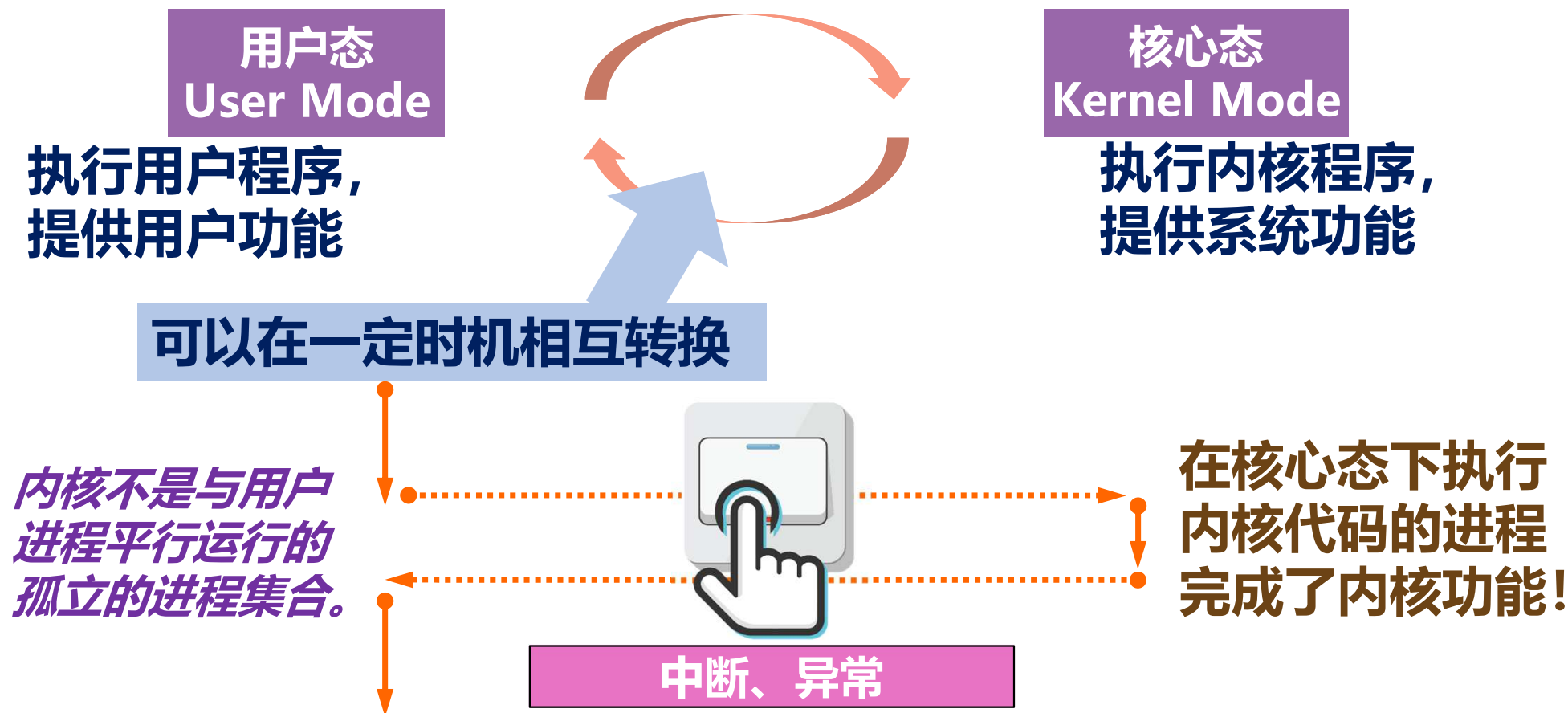
异常/陷入



UNIX异常处理



概况

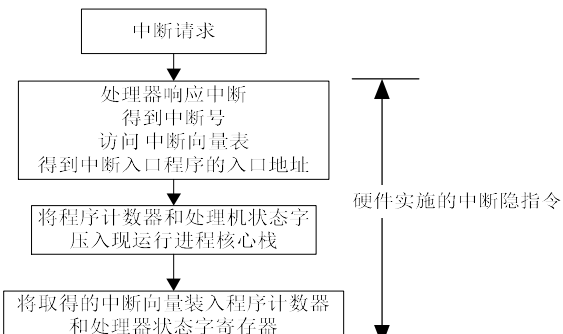




UNIX异常处理



硬件实施中断隐指令

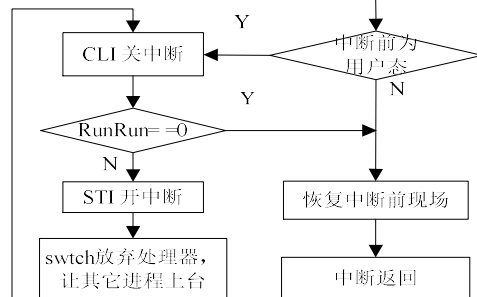


保存中断现场

切换至核心态

异常处理程序

异常入口程序



```
void Exception::DivideErrorEntrance()
```

```
{  
    SaveContext();  
    SwitchToKernel();
```

```
    CallHandler(Exception, DivideError);
```

```
    RestoreContext();  
    Leave();  
    InterruptReturn();  
}
```

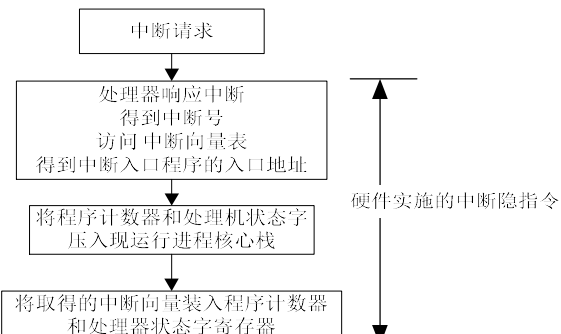
以除数为0的异常为例：



UNIX异常处理



硬件实施中 断隐指令

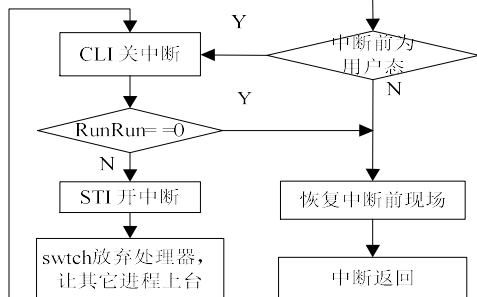


保存中断现场

切换至核心态

异常处理程序

异常入口程序



```
void Exception::DivideErrorEntrance()
{
    SaveContext();
    SwitchToKernel();

    CallHandler(Exception, DivideError);

    RestoreContext();
    Leave();
    InterruptReturn();
}
```

以除数为0的异常为例:

与中断的主要区别:

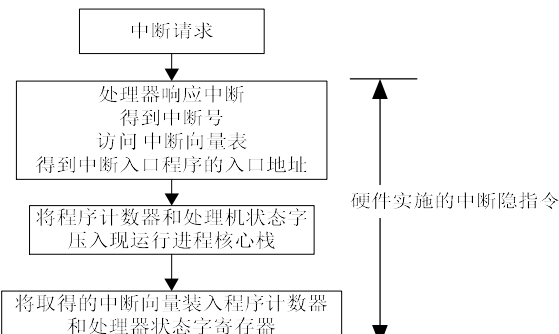
1. 没有EOI



UNIX异常处理



硬件实施中 断隐指令



保存中断现场

切换至核心态

异常处理程序

异常入口程序

恢复中断前现场

中断返回

```
void Exception::DivideErrorEntrance()
{
    SaveContext();
    SwitchToKernel();

    CallHandler(Exception, DivideError);

    RestoreContext();
    Leave();
    InterruptReturn();
}
```

以除数为0的异常为例：

与中断的主要区别：

1. 没有EOI
2. 没有例行调度

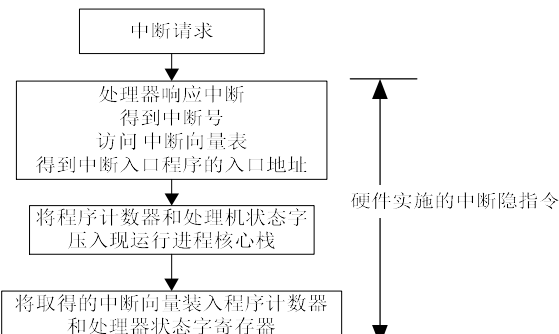


UNIX异常处理



概况

硬件实施中 断隐指令



```
void Exception::DivideErrorEntrance()
{
    SaveContext();
    SwitchToKernel();

    CallHandler(Exception, DivideError);

    RestoreContext();
    Leave();
    InterruptReturn();
}
```

以除数为0的异常为例：

异常入口程序

异常处理程序

与中断的主要区别：

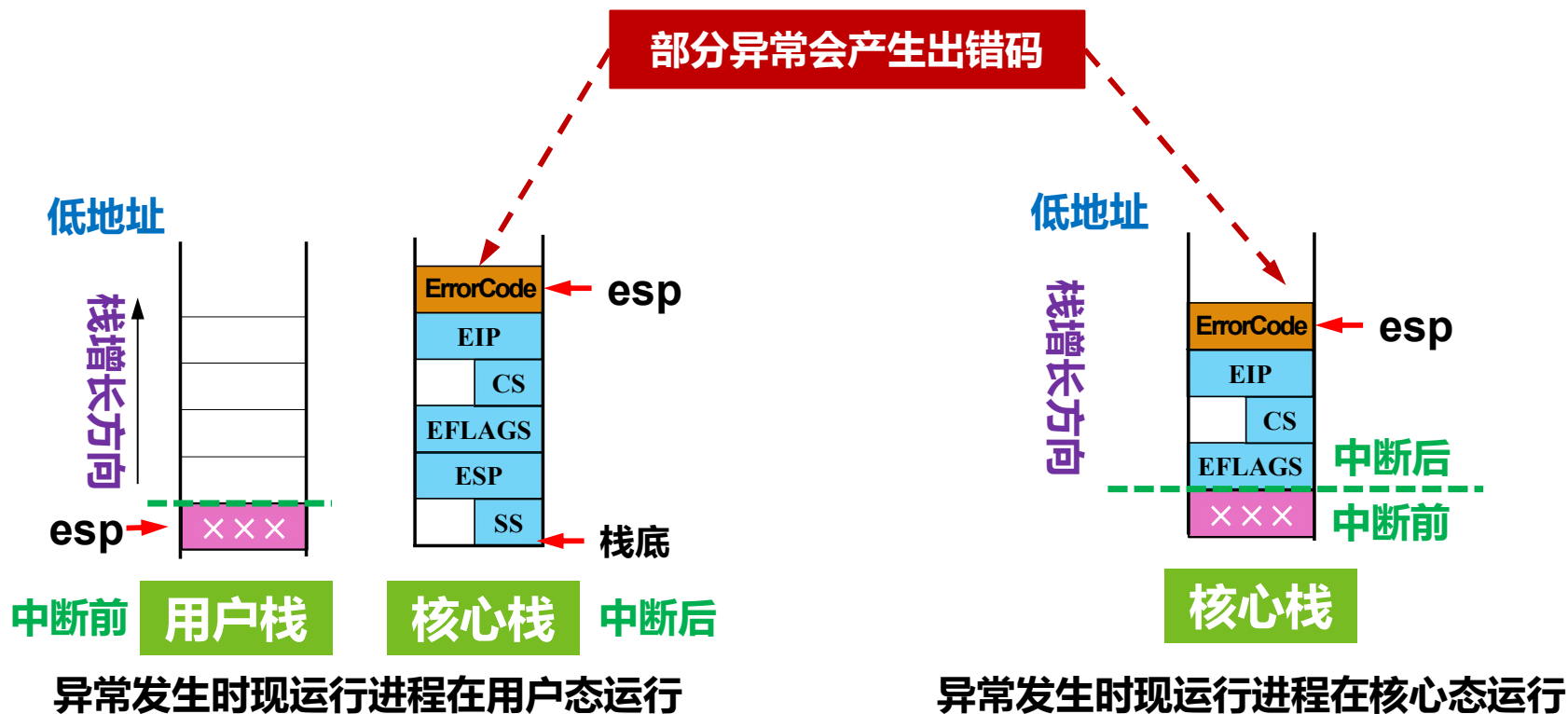
1. 没有EOI
2. 没有例行调度
3. 中断隐指令形成的硬件现场



UNIX异常处理



异常处理的详细流程



中断隐指令实施现场保护后的堆栈状态

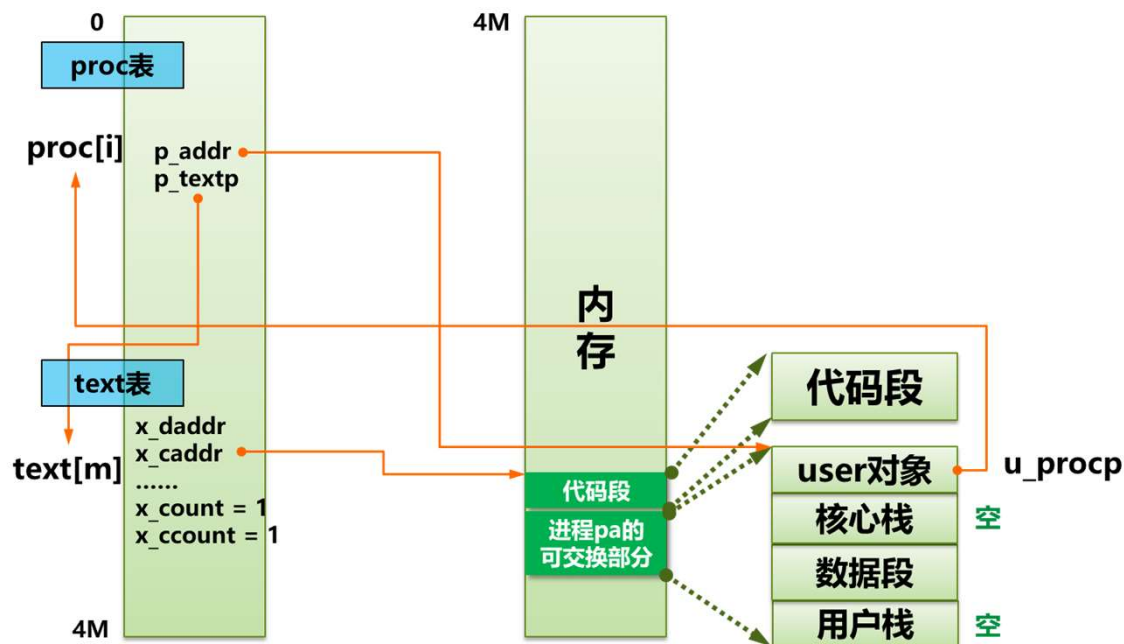


再看下面这个简单的程序



```
#include <fcntl.h>
char buffer[2048];
int version = 1;
main( argc, argv)
int  argc;
char *argv[];
{
    int a, b;
    .....;
    sum(a, b);
    exit(0);
}
int sum( var1, var2)
int var1, var2;
{
    int count;
    count = var1 + var2;
    return(count);
}
```

第一次执行..... 创建进程pa, pa的图象:





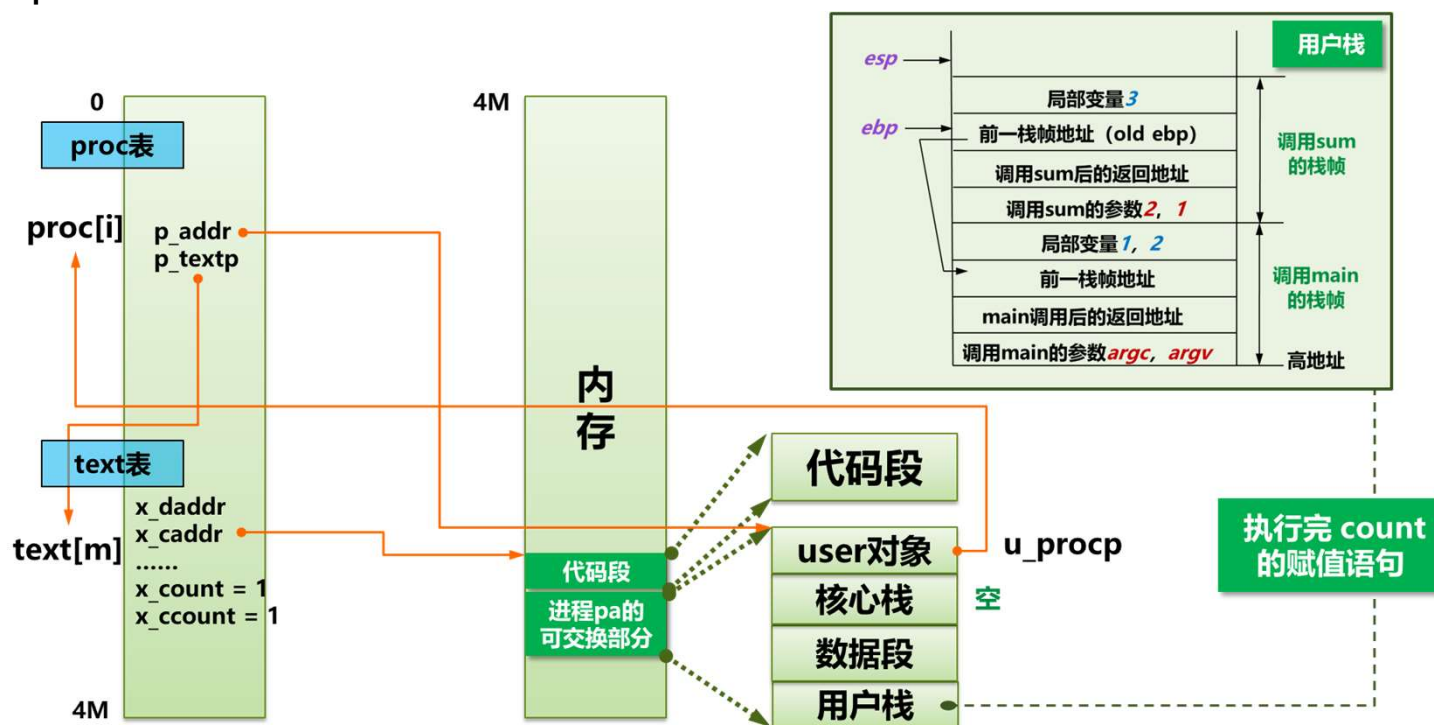
再看下面这个简单的程序



```
#include <fcntl.h>
char buffer[2048];
int version = 1;
main( argc, argv)
int  argc;
char *argv[];
{
    int a, b;
    .....;
    sum(a, b);
    exit(0);
}
int sum( var1, var2)
int var1, var2;
{
    int count;
    count = var1 + var2;
    return(count);
}
```

第一次执行..... 创建进程pa, pa的图象:

pa执行完赋值语句后:

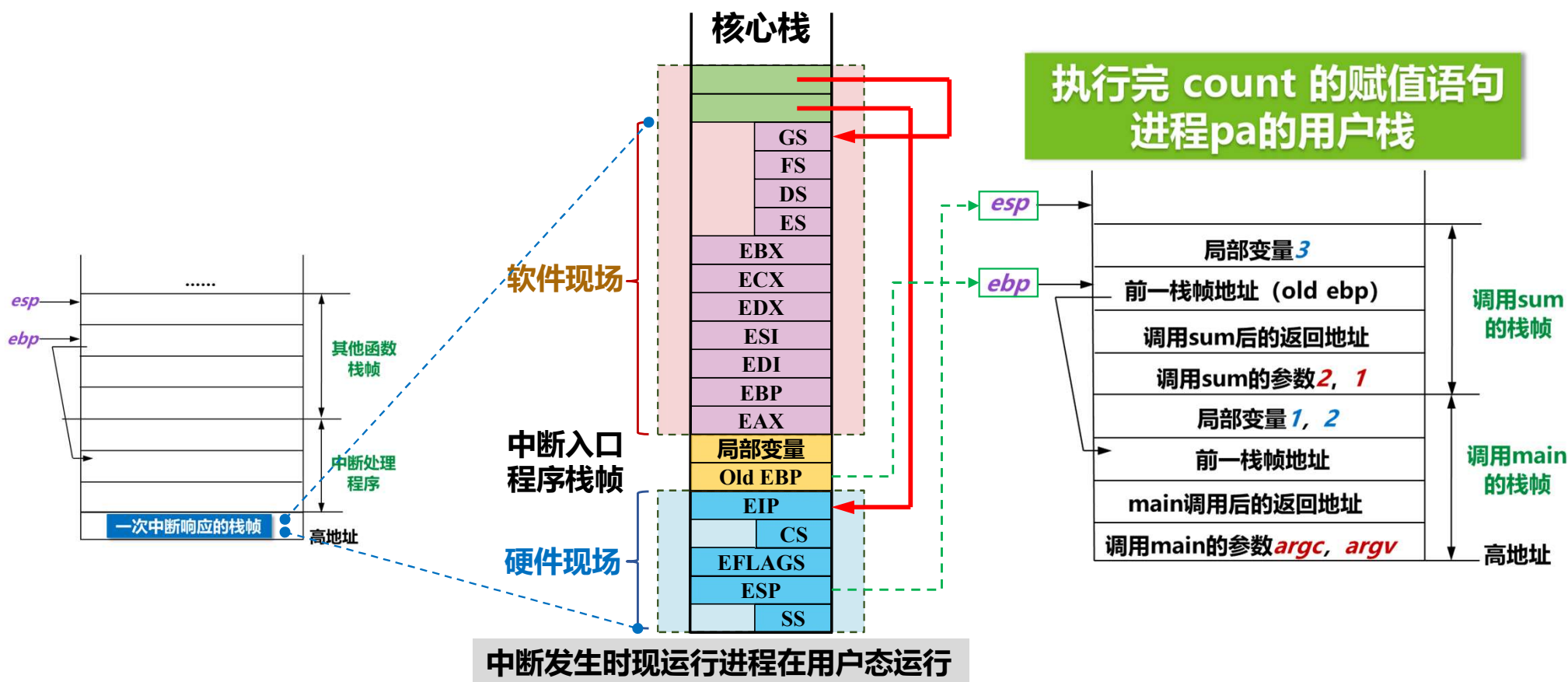




再看下面这个简单的程序



如果这时，有一磁盘的中断请求，.....

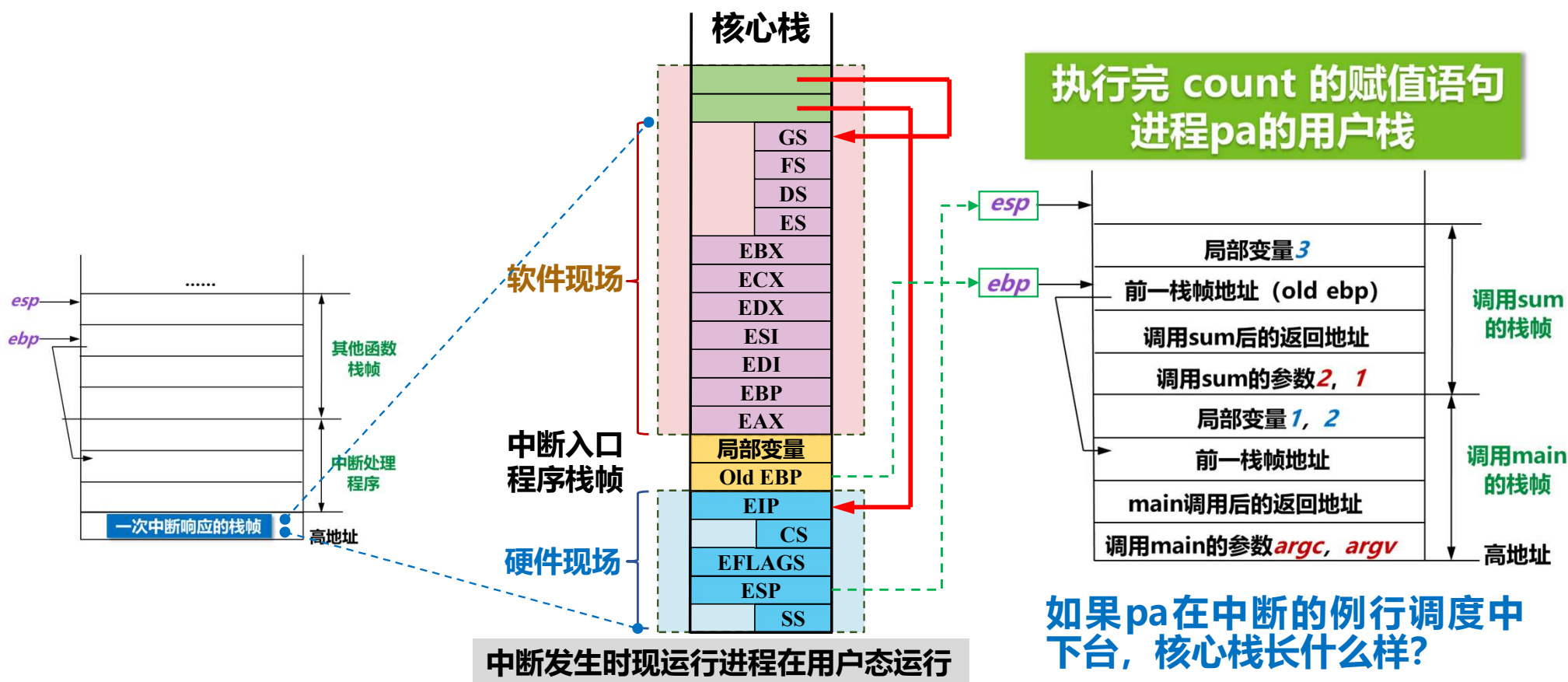




再看下面这个简单的程序



如果这时，有一磁盘的中断请求，.....

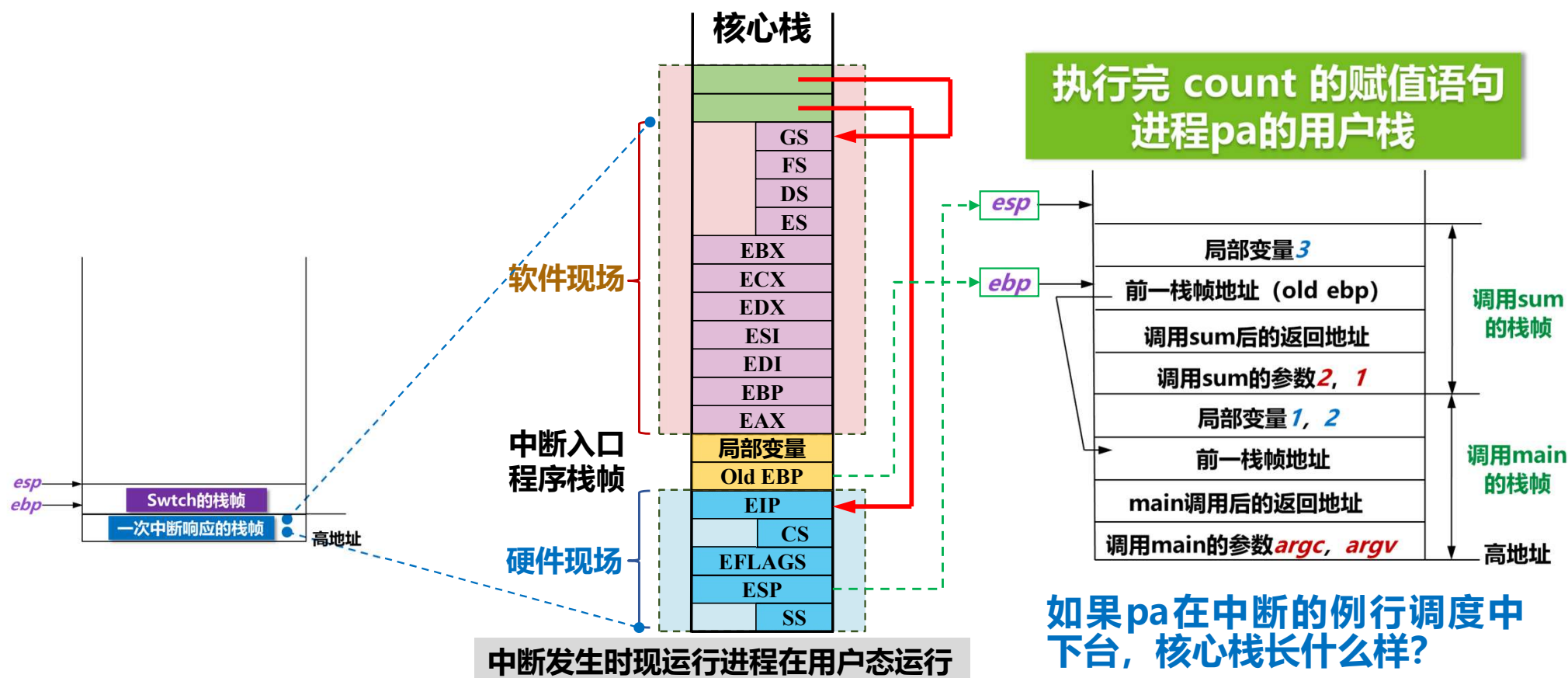




再看下面这个简单的程序



如果这时，有一磁盘的中断请求，



如果pa在中断的例行调度中下台，核心栈长什么样？



本节小结



- 1 UNIX中断的处理过程
- 2 UNIX中断处理过程中核心栈的变化

阅读教材：84页 ~ 103页

阅读代码：TimeInterrupt.cpp中Time::TimeInterruptEntrance()代码



E04: 并发进程 (UNIX中断)